

HOPPER

A Satoshi-Native Game Protocol on Bitcoin

One satoshi. One life.
The chain is the referee.

Wire version `0x08`

Specification 3.0 · October 2026

The HOPPER contributors

Abstract

HOPPER is a game played by a satoshi. A frog is an ordinal sat whose public life is written into Bitcoin itself: a mint inscription creates it, each crossing publishes a 55-byte state inscription on the same sat, and the direction carried in that inscription is judged against the one direction the next block wraps to. Custody is enforced by a four-leaf Taproot output with a nothing-up-my-sleeve internal key. Three crossing leaves, one per direction, each check the owner's signature and carry the protocol envelope; the fourth leaf is an ordinary owner signature for transfers. The v8 fold verifies every crossing body against the raw transaction that published it; if a proof is missing, the art says so rather than guessing. The road's difficulty runs in generations: it rises from five crossings to a ceiling of twenty-five and then resets. One released v7 Hopper is grandfathered for custody and play. No server signs, no external origin is needed to render the frog, and the indexer is a cache, not an authority.

This paper specifies the wire format, the generation schedule, the block-wide wrap, the tapscript tree and transaction shapes that play the game, and the standalone indexer and JavaScript wallet that implement them. It is an exposition of the normative specification in `SPEC.md`; where the two documents differ, `SPEC.md` governs.

Keywords: Bitcoin, ordinals, inscriptions, Taproot, tapscript, ordinal theory, self-custody, PWA.

Normative source. The repository specification `hopper-protocol/SPEC.md` is the single normative source for the consensus rules, byte layouts, constants, API surface and configuration described here. Code snippets are illustrative; byte strings, constants and acceptance conditions are quoted from that document.

The historical mint magic `FRG` and the JSON keys `frog`/`frogger` are retained because they are already committed to Bitcoin. The current state magic is `FRG8CR0SS`; the legacy `FRG7CR0SS` is accepted only for the one grandfathered Hopper. The interface name is HOPPER; the crossing action is `H0P` in the interface and `cross` on the wire.

Contents

Abstract **i**

1 Introduction **1**
1.1 Design goals . . . 1
1.2 Why a satoshi? . . . 1
1.3 What is new in v8 . . . 2
1.4 Contributions and outline . . . 2

2 Model and notation **3**
2.1 Bitcoin, sats and ordinal ranges . . . 3
2.2 Taproot and tapscript . . . 3
2.3 The ord envelope . . . 4
2.4 Trust model . . . 4
2.5 Notation . . . 4

3 Parameters and the road **4**
3.1 Constants . . . 4
3.2 Generations and the road's difficulty . . . 5
3.3 The road's wrap . . . 6
3.4 Parameter commitment . . . 7
3.5 Outcome timing . . . 7

4 Inscriptions and the fold **7**
4.1 The mint envelope . . . 7
4.2 The ord envelope . . . 8
4.3 The mint body: a self-contained proof-checking fold document . . . 8
4.4 The crossing body . . . 9
4.5 The fold algorithm . . . 10
4.6 The one grandfathered v7 Hopper . . . 12

5 Game semantics **12**
5.1 The frog record . . . 12
5.2 Mint . . . 13
5.3 Hop . . . 13
5.4 Settlement timers . . . 14
5.5 Transfer and burn . . . 15
5.6 Events . . . 15
5.7 Reorganizations . . . 16
5.8 A crossing in six acts . . . 16

6 Custody: the Taproot tree that plays the game **16**
6.1 The state output . . . 16
6.2 The cross leaf . . . 17

6.3 Control blocks and witnesses . . . 18
6.4 The two-phase mint . . . 18
6.5 Action transactions . . . 19
6.6 Previews and the ordinary send . . . 20
6.7 Sat assignment . . . 20
6.8 Addresses, discovery and history . . . 21

7 The standalone indexer 21
7.1 Crate boundaries . . . 21
7.2 Ingest . . . 22
7.3 State, snapshots and the state root . . . 23
7.4 HTTP API . . . 23
7.5 Configuration . . . 24
7.6 Seeding . . . 24
7.7 Rejections . . . 24

8 The wallet and the PWA 25
8.1 Keys and the vault . . . 25
8.2 Chain access . . . 26
8.3 Screens and interaction . . . 27
8.4 Sound . . . 27
8.5 Emergency descriptor export . . . 28
8.6 Wallet safety invariants . . . 29
8.7 The user’s trust boundary . . . 29

9 Security analysis 30
9.1 Threats and mitigations . . . 30
9.2 Properties . . . 31
9.3 Limitations and honest assumptions . . . 31
9.4 Why the extra transaction is unavoidable . . . 32
9.5 Compatibility notes . . . 32

10 Validation 32
10.1 Reference vectors . . . 33
10.2 Conformance discipline . . . 34

11 Related work 34

12 Conclusion 35

A The generation schedule 35

B Deployment quick reference 36

Reading guide. Sections 3–6 are the consensus core. Sections 7 and 8 describe infrastructure that can be replaced without changing the protocol. Section 9 states what the construction does and does not protect against.

1 Introduction

Bitcoin [1] is usually described as a ledger of coins. Ordinal theory gives that ledger a second reading: every satoshi has a number, sats move through transactions in first-in-first-out order, and a particular sat can therefore be followed across the chain. HOPPER uses that identity directly. A frog is not a token contract, a database row, or an entry in a server's table. A frog is one satoshi, and its public life is the sequence of transactions that carry that satoshi.

The game is small enough to state in one paragraph. A player mints a frog by inscribing a self-contained art document on a sat. A one-time prepare may move the frog from its mint output into the custody output. To move the frog forward, the owner spends the sat through one of three Taproot leaves and publishes a 55-byte state inscription naming the direction it picked: left, straight or right. The crossing confirms at height T. At block $T + 1$ the block hash wraps to exactly one direction for the whole block, and every frog whose crossing confirmed in T that picked that direction dies at $T + \kappa$. A frog that survives enough crossings reaches *home* and becomes immortal. A plain transfer moves the sat without publishing anything and without changing the frog's state. The chain is the referee, and the owner chooses *when* to hop and *which way*, never whether the road agrees.

1.1 Design goals

The construction is organized around four goals.

State on the sat, not beside it.
Every state transition is an ord inscription on the frog's own sat. A stock `ord` viewer can reconstruct the frog from chain data alone, without a HOPPER server. The indexer accelerates this; it is not the source of truth.

Custody by Bitcoin Script.
A frog in custody sits in a Taproot output whose internal key is a nothing-up-my-sleeve point. There is no key-path spend. Three leaves sign a crossing, one per direction, each carrying the state envelope; the fourth is a plain owner signature for transfers. A valid crossing is therefore both a signature and a protocol message, and the direction is bound into the leaf that authorized it.

Outcomes the author cannot choose.
At block $T + 1$ the block hash selects one direction for the entire block. A crossing dies exactly when the direction committed in its body equals that wrap. The block hash does not exist when the crossing is signed, and the function is public. No author, owner or indexer can declare an outcome. A block producer can influence the candidate hash or withhold a block; that is miner influence, not player agency, and the protocol does not pretend otherwise.

An implementation small enough to audit.
The consensus surface is a 55-byte body, a 34-byte send leaf, a 161-byte crossing leaf, a 36-byte parameter encoding, and three integer functions: generation, required crossings, and the block wrap. The reference implementation is split into a pure protocol crate, a pure state machine, and a network-facing API.

1.2 Why a satoshi?

A satoshi is the smallest unit Bitcoin can own, and ordinal theory gives it a stable name. This has three consequences that matter for a game.

First, ownership is UTXO ownership. There is no separate balance to reconcile and no issuer-controlled registry that can reassign a frog. Second, the sat carries its history. A state inscription is not attached to a mutable object; it is attached to the same sat as the mint, so the child relationship is cryptographically anchored by the satpoint as well as by the parent tag. Third, the identity survives transfers. A frog can be sent

to an ordinary address, held, and later brought back into custody without losing its sat or its accumulated history.

The price of this design is that a frog is only as movable as the UTXO that contains it. A wallet must know which ordinal ranges a coin carries before it spends that coin. The protocol makes this requirement explicit: a coin whose ranges cannot be attributed is still counted in the balance, but it is shown as unspendable rather than spent blindly.

1.3 What is new in v8

The game rules introduced by wire version `0x07` still shape everything that follows. Wire version `0x08` adds a proof-bound envelope, a proof-checking fold and a single legacy exception.

1. **The road asks a question, not a die roll.** Each crossing body carries a direction: left, straight or right. The next block's hash wraps to one direction and applies it to every crossing in the previous block. There is no per-Hopper death table and no per-Hopper randomness.
2. **Custody has four leaves.** Three crossing leaves, one per direction, plus one send leaf. Choosing a direction selects a leaf whose bytes were fixed when the state output was created.
3. **Difficulty runs in generations.** A generation starts at five required crossings, adds one every 144 blocks up to twenty-five, holds for 3,600 blocks, and resets. Each generation stamps its frogs `G1`, `G2`, and so on; the label is fixed for life, the target is not.

Version 8 changes the bytes and the verification story, not the road. The mint body is now `live_v8_html`, the state magic is `FRG8CROSS`/`0x08`, and the embedded fold checks every body-valid child against the raw transaction that published it. A proof that is missing or false makes the fold report *unverified*, rendered as `NAME ⚌ ---`; it never prints a count or claims `HOME` on incomplete evidence. The fold ignores a body-invalid child, but a body-valid child with a failed proof is a hard stop.

Exactly one released v7 Hopper is carried across the cut. Its sat is the compiled `GRANDFATHERED_SAT`. For that sat only, the v7 mint envelope and digest, the `V7` body and the v7 `FRG7CROSS`/`0x07` state path are accepted, because already-confirmed v7 custody cannot be rewritten. Every other sat must use v8. The released v7 card's fold predates the proof check; it is grandfathered for custody and play, not as an authoritative art oracle. A proof-checked view comes from the v8 app or indexer.

The mint reveals a self-contained HTML document whose canonical fold reads only relative `/r/` and `/content/` endpoints on the viewer's own ord node. A crossing publishes an `application/octet-stream` body containing the frog's sat, mint identity and chosen direction, together with an ord parent tag. The first state output must be created by a later transaction — the first transfer or a one-time prepare — because the mint transaction cannot embed its own txid; after that, each crossing spends the previous state output and creates the next one.

1.4 Contributions and outline

This paper makes the following contributions.

1. A precise inscription format for mint and crossing, including the ord envelope, the parent encoding, the canonical PNG encoder, the single accepted v8 mint-body digest and its one legacy exception (Section 4).
2. A consensus state machine for mint, hop, settlement, transfer and burn, with a parameter commitment, a generation schedule and a block-wide wrap (Sections 3 and 5).
3. A four-leaf Taproot construction that binds the protocol message and the chosen direction to the spend, with exact leaf scripts, control blocks, witnesses and transaction shapes (Section 6).

4. A standalone Rust indexer with deterministic state, snapshots, a reorg window and a documented HTTP API (Section 7).
5. A JavaScript wallet/PWA that keeps keys in a WebCrypto vault, reproduces the interface and sound, and exports emergency descriptors for its staged outputs (Section 8).
6. A proof-checking fold, a fail-closed unverified state and the single grandfathered-v7 rule, plus a security analysis and a conformance-oriented test plan (Sections 4, 9 and 10).

Section 2 fixes notation and the trust model. Section 3 states the constants, the generation schedule and the wrap. Section 4 specifies the envelopes. Section 5 gives the state machine. Section 6 gives the tapscript tree and transaction shapes. Sections 7 and 8 describe the reference infrastructure. Section 9 analyses attacks and limits, and Section 10 lists the test obligations. The appendices reproduce the generation schedule and the deployment constants.

The one-sentence rule

A Hopper is a satoshi. Its state is the ordered set of valid HOPPER inscriptions on that satoshi. Its fate is a function of the chain, not of its owner.

2 Model and notation

2.1 Bitcoin, sats and ordinal ranges

A Bitcoin transaction consumes previous outputs and creates new ones. Each output has a value in satoshis. Ordinal theory assigns the sats of the inputs to the outputs in order, so an output can be described by a list of half-open intervals $[a, b)$ of ordinal numbers. A coin's *first sat* is the first sat in its first interval. Transfers preserve this order: sats are spent first-in-first-out, and a later output cannot jump ahead of an earlier one.

Two properties follow and are used throughout the protocol.

1. A frog's anchor sat is the first sat of the output that carries it. In a mint, that is the first sat of the commit output; after a spend, it must be at offset zero of output zero.
2. A coin that the indexer has never seen move has no true ordinal assignment. It receives a deterministic *shadow* range so that FIFO placement remains well-defined, but a shadow range is not ownership. A mint whose anchor sat is a shadow is rejected as `MintAnchorUntracked`.

The protocol never relies on a floating-point sat number or on a database identifier. The sat is the identity, and the satpoint (transaction id and output index) is its current location.

2.2 Taproot and tapscript

A Taproot output commits to an internal key P and a Merkle root of a script tree. The output key is $Q = P + tG$ for a tweak t derived from the tree. A key-path spend signs with the tweaked key; a script-path spend reveals one leaf, the leaf script, and a control block that proves the leaf's position in the tree. BIP-341 [5] defines the output key and control block; BIP-342 [6] defines the tapscript signature hash used by `OP_CHECKSIG`.

HOPPER uses the script path exclusively. The internal key is the fixed nothing-up-my-sleeve point `NUMS_XONLY`, for which no discrete log is known. There is no key-path spend, ever. The tree has four leaves at depth two: three *cross* leaves, one per direction, each checking the owner's signature and carrying the state envelope for that direction, and one *send* leaf that checks only the owner's signature. The exact scripts and their branch structure are given in Section 6.

2.3 The ord envelope

An ord inscription is a tapleaf containing an envelope of the form

```
OP_FALSE OP_IF "ord" (tag, value)... 0x00 (body chunks) OP_ENDIF.
```

The protocol identifier is the three-byte push `"ord"`. Tag `1` is the content type, tag `3` is a parent, and the empty push `0x00` marks the start of the body. Body chunks are limited to 520 bytes and are concatenated in order. The indexer's parser mirrors ord's decoder, including ord's minimal-index rule for parent tags.

HOPPER writes only the fields it needs: a content type and a body, plus one parent tag on a crossing. It never writes a pointer tag. A pointer would move the inscription off the anchor sat, which would break the identity between the frog and its state; a mint inscription carrying a pointer is rejected.

2.4 Trust model

The protocol distinguishes four roles.

Table 1: *What each component is trusted to do.*

Component	Trusted for	Not trusted for
Bitcoin consensus	Transaction validity, ordering, block hashes, finality	Application-level meaning
ord viewer	Parsing ord envelopes and serving `/r/` content	Knowing HOPPER's rules
HOPPER indexer	Convenience: fast discovery, API, event log	Custody, signatures, art truth, the outcome of a future block
Wallet	Key custody, transaction construction, user intent	Chain history it has not checked

The art document is deliberately independent of all four. It embeds the fold source and reads only relative endpoints on the viewer's own ord node. The static band states only what the mint transaction can prove: the name, a generation hint, and an unknown crossing count. It never prints a number. The live fold may render a count only after it has verified every body-valid child's raw transaction; a missing or false proof yields the *unverified* band, which prints no count and never claims `HOME`. Only a settled frog gets `RIP`. If the viewer refuses every fetch or runs with JavaScript disabled, the static band stays and does not assert that the frog is alive. The indexer can be unavailable without changing what a frog is.

The adversary is assumed to control network delivery, to be able to censor or reorder transactions before confirmation, and to attempt to spend outputs it does not own. It is not assumed to break secp256k1, SHA-256, BIP-340 signatures or Bitcoin consensus. Reorganizations are treated as a normal chain event, not as an attack.

2.5 Notation

All integer encodings in this paper are little-endian unless a field is explicitly described as display-order hex. Transaction ids and block hashes are shown to humans in reversed (display) order, as explorers do; the wrap uses the internal order.

3 Parameters and the road

3.1 Constants

The protocol is parameterized by a small, compiled-in set. `genesis` selects the deployment's schedule; every other schedule value is frozen. The grandfathered sat is a separate compiled exception, not a schedule parameter. The reference deployment uses the values in Table 3.

Table 2: *Notation used in the remainder of the paper.*

Symbol	Meaning
G	Generation genesis height; the only deployment value
sat_g	The one grandfathered v7 sat, or 0 when none
b	Required crossings on the first block of a generation
gi	Blocks per extra required crossing
c	Ceiling, the hardest the road ever gets
h_{hold}	Blocks spent at the ceiling before the reset
L	Generation length, $(c-b)\mathrm{gi}+h_{\mathrm{hold}}$
$S(k)$	First height of generation k (1-based)
$\gamma(h)$	Generation number at height h
$r(h)$	Required crossings at height h
$w(B)$	The direction block B wraps to, in $\{0,1,2\}$
T	Confirmation height of a crossing
B	Outcome block, $B=T+1$
H_B	32-byte hash of block B, internal byte order
κ	Crossing finality, six blocks
v	Wire version: `0x08`, or `0x07` for the grandfather
sat	The frog's ordinal number (a `u64`)
dir	Chosen direction: 0 left, 1 straight, 2 right

Table 3: *Protocol constants for wire version `0x08`.*

Name	Value	Meaning
`wire version`	`0x08`	The current wire version
`legacy wire version`	`0x07`	Grandfathered sat only
`mint magic`	`"FRG"`	Mint envelope prefix
`mint action`	`0x01`	Mint action byte
`state magic`	`"FRG8CROSS"`	9-byte crossing body prefix
`legacy state magic`	`"FRG7CROSS"`	Grandfathered sat only
`state body length`	55 bytes	Fixed crossing body size
`grandfathered sat`	`1,965,983,893,977,665`	One v7 Hopper; `0` = none
`activation height`	974,034	Frozen inscription cutover
`direction encoding`	`0,1,2`	Left, straight, right
`wrap function`	`u32LE(H)[0..4] mod 3`	The block's one direction
`crossing leaves`	3 + 1 send	One leaf per direction, plus send
`cross_finality`	6 blocks	Settlement delay after T
`card_value`	25,000 sats	Committed card value
`anchor output value`	1,000 sats	Value of a frog-bearing output
`dust (P2WPKH)`	546 sats	Dust threshold for change
`dust (P2TR)`	330 sats	Dust threshold for Taproot change
`NUMS internal key`	`50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0`	

3.2 Generations and the road's difficulty

The required number of crossings runs in generations. A generation starts at b crossings, adds one every gi blocks, reaches the ceiling c, holds there for h_{hold} blocks, and then resets to b. The values are

$$b=5, \qquad \mathrm{gi}=144, \qquad c=25, \qquad h_{\mathrm{hold}}=3{,}600, \tag{1}$$

so the generation length is

$$L=(c-b)\mathrm{gi}+h_{\mathrm{hold}}=(25-5)\cdot 144+3{,}600=6{,}480. \tag{2}$$

Generation k (1-based) starts at

$$S(k) = G + (k-1)L, \tag{3}$$

and owns the half-open interval $[S(k), S(k+1))$. Its number at height h is

$$\gamma(h) = \begin{cases} 1, & h < G, \\ 1 + \left\lfloor \dfrac{h-G}{L} \right\rfloor, & h \geq G, \end{cases} \tag{4}$$

and the requirement is

$$r(h) = \begin{cases} b, & h < G, \\ \min\left(b + \left\lfloor \dfrac{h - S(\gamma(h))}{\mathrm{gi}} \right\rfloor,\ c\right), & h \geq G. \end{cases} \tag{5}$$

A generation therefore starts at five required crossings, adds one every 144 blocks, reaches twenty-five at $S(k) + 2{,}880$, holds twenty-five for 3,600 blocks, and restarts at five. The indexer rejects a published parameter block whose `generation_len` is not $(c-b)\mathrm{gi} + h_{\mathrm{hold}}$.

The function is a *sawtooth*, not monotonic. The reset at a generation boundary is deliberate and load-bearing; it must not be "smoothed". The target used by a crossing is $r(T)$, where T is the crossing's confirmation height. It is frozen when the crossing confirms and does not follow a later reset.

A Hopper's *generation* is the generation of its mint confirmation height, $\mathrm{gen} = \gamma(\mathrm{mint_height})$. It is immutable for the frog's life and is what the art renders as `G1`, `G2`, and so on. A frog born in G1 can still hop in G3; only its label is fixed, never its target.



Figure 1: *Two generations of the schedule. Each climbs from five to twenty-five over 2,880 blocks, holds the ceiling for 3,600 blocks, then resets. The sawtooth is intentional.*

3.3 The road's wrap

There is no per-Hopper death table and no per-Hopper randomness. Each block has exactly one direction, and every crossing that confirmed in the previous block is judged against it:

$$w(B) = \mathrm{LE}_{32}\left(H_B^{\mathrm{int}}[0..4]\right) \bmod 3, \qquad \text{dead} \iff \mathrm{dir} = w(T+1). \tag{6}$$

Here H_B^{int} is the 32-byte block hash in internal byte order — the reverse of the hex an explorer or ord returns — and LE_{32} reads its first four bytes as a little-endian `u32`. dir is the direction the player chose, carried in the crossing body (Section 4). Direction 0 is left, 1 straight, 2 right.

Normative consequences:

- The wrap is **global**. Every Hopper whose crossing confirmed in block T is judged against the same $w(T+1)$. If the block wraps right, every frog in that block that hopped right dies and every other frog in that block survives.
- The wrap is a function of the block hash alone — not of the sat, the owner, the name, the target or the number of frogs in the block — so an ordinary player cannot choose or predict its own outcome. A block producer can influence the candidate hash or withhold a block; that is miner influence, not player agency.
- Since $2^{32} = 3 \cdot 1{,}431{,}655{,}765 + 1$, the modulo is biased by one part in 2^{32} toward left. The bias is accepted. An implementation must use this exact integer; it must not substitute a wider word, a rejection scheme, or a hash of the hash.

3.4 Parameter commitment

The parameters are committed to as bytes, so two nodes with different schedules cannot share a snapshot or a state root. The fields, in order, are `genesis`, `base`, `growth_interval`, `ceiling`, `hold`, `generation_len`, `cross_finality` and `card_value`. The canonical encoding is the 36-byte concatenation

$$\begin{aligned}\mathrm{canon}(P) = {} & \mathrm{LE}_{32}(G) \,\|\, \mathrm{LE}_{32}(b) \,\|\, \mathrm{LE}_{32}(\mathrm{gi}) \,\|\, \mathrm{LE}_{32}(c) \\ & \|\, \mathrm{LE}_{32}(h_{\mathrm{hold}}) \,\|\, \mathrm{LE}_{32}(L) \,\|\, \mathrm{LE}_{32}(6) \,\|\, \mathrm{LE}_{64}(25{,}000),\end{aligned} \tag{7}$$

and the parameter hash is

$$H_{\mathrm{params}} = \mathrm{SHA256}(\mathrm{canon}(P))\,. \tag{8}$$

For the reference deployment,

`d466f8db3efa330dc92444274675804e47ec76b62393eeae88063ef764bcf72f`.

A wallet refuses to mint or hop when the indexer's published H_{params} differs from its own. It also refuses an indexer that does not answer the parameter gate, or whose `network` does not match the wallet's network. Missing identity is not permission to sign. ord remains the canonical source for sat ranges and inscription validity; the HOPPER indexer is for discovery and the leaderboard, never for overriding ord. A snapshot whose stored hash differs is a hard error, never a migration.

3.5 Outcome timing

$w(T+1)$ is learned when block $T+1$ is mined; the outcome is applied at $T+\kappa = T+6$. Nothing about a Hopper's state changes in between, and a reorg that replaces $T+1$ replaces the wrap. The wrap is recorded once per block and applied to every crossing in the previous block, so the game is decided by the chain at block granularity, not by a per-frog draw.

4 Inscriptions and the fold

A frog touches Bitcoin in two kinds of inscription: the mint document and the crossing state body. Both use ord's envelope grammar. The mint is a self-contained HTML document; the crossing is 55 bytes of binary state. The mint envelope in the OP_RETURN output names the seed and name; the mint inscription carries the art. The indexer validates both, because the inscription is the part a stock ord viewer can read without HOPPER software.

4.1 The mint envelope

The mint transaction carries one OP_RETURN output whose push is the following byte string.

Table 4: *The `0x08` mint envelope. The total length is at most 80 bytes. The `0x07` version is accepted only for the grandfathered sat.*

Offset	Length	Field	Value
0	3	magic	`"FRG"`
3	1	version	`0x08` (legacy `0x07` only for the grandfather)
4	1	action	`0x01` (MINT)
5	4	seed	`u32` little-endian
9	1	name length	0 to 38
10	*n*	name	UTF-8, *n* bytes

Exactly one `FRG` envelope is allowed per transaction. A malformed or duplicated envelope invalidates the protocol message: the transaction still moves sats, but it is not a mint. The same seed and name appear in the mint inscription reveal; the indexer validates both. This redundancy is intentional. The OP_RETURN makes the mint cheap to index; the inscription makes it readable by ord alone.

4.2 The ord envelope

The mint reveal's tapleaf contains ord's standard envelope:

Listing 1: *Mint inscription envelope (grammar).*

```
OP_FALSE OP_IF
  "ord"                              ; 3-byte protocol id
  0x01 "text/html;charset=utf-8"     ; tag 1: content type
  0x00                               ; body tag
  <body, chunked at 520 bytes>       ; the fold document
OP_ENDIF
```

The crossing envelope is the same grammar with a binary content type, a parent tag and a fixed body:

Listing 2: *Crossing inscription envelope (grammar).*

```
OP_FALSE OP_IF
  "ord"
  0x01 "application/octet-stream"
  0x03 <mint_txid:32 || minimal-LE mint_index>   ; tag 3: parent
  0x00 <55-byte body>
OP_ENDIF
```

The parent value is the mint transaction id in internal byte order, followed by the mint output index. Index zero is encoded as the bare 32-byte txid; a non-zero index is appended as a minimal little-endian integer. Ord's rule is strict: a one-to-three-byte index whose last byte is `0x00` is non-minimal and rejected. The parent tag is mandatory on a crossing; it is what makes `/r/children` work and what lets a viewer discover the crossing history of a mint.

4.3 The mint body: a self-contained proof-checking fold document

The mint body is the fold document returned by `live_v8_html` for a `(seed, sat, name, gen_hint)` tuple, with content type `text/html;charset=utf-8`. It has five properties:

1. **No external origin.** There is no `http://`, `https://`, bare host, or protocol-relative URL anywhere in the document. This is grep-enforced in the implementation.
2. **Two inline sprites.** The document embeds two `...` 16×16 images, one idle and one dead.
3. **A static name band.** The card carries the Hopper's name, its generation badge (`G1`, `G2`, …) and the road mark with an unknown crossing count, for example `NAME` · ⚌ –. It never prints a number.

4. **An embedded parameter block.** The document contains

```
V8={sat, genHint, name, [grandfatheredSat,] p:{g,b,gi,cap,hold,gl,fin}}
```

   where `g` is the genesis height, `b` the base, `gi` the growth interval, `cap` the ceiling, `gl` the generation length and `fin` the finality. `grandfatheredSat` is emitted only when the compiled value is non-zero.

5. **The canonical fold source.** The embedded `live_v8.js` reads only relative `/r/...` and `/content/...` endpoints on the viewer's own ord node.

There is no `STATE FROM THE CHAIN` caption in v8. The static band states only what the mint transaction itself can prove: the name, the generation hint from the intended mint height, and that the count is unknown. The live band is authoritative only when the fold has verified every body-valid child against the raw transaction that published it. If a proof is missing or false, the band is `NAME · 🛣` – and no count or `HOME` is claimed. When the proof holds:

```
alive:  NAME🛣 12     dead:  RIP NAME🛣 12
home:  NAME🛣 25 HOME
```

The count is `dist`, the number of completed crossings, and it is shown for the dead as well. `RIP ` (with its trailing space) is prepended to the name only when the fold has settled the frog as `dead`. The road glyph is illustrative; the canonical text form is `NAME🛣 N`, and that is what tests compare. The implementation uses the road emoji U+1F6E3 between the name and the count; this paper draws the mark instead, because the body font carries no emoji glyph.

`gen_hint` is the generation of the height the wallet expects the mint to confirm at. It is a display hint, never consensus: the indexer derives `gen` from the confirmed `mint_height` and ignores `gen_hint`. The live fold shows the authoritative `G` when it can read the mint inscription's height and falls back to the hint when it cannot.

The PNG encoder is canonical and deterministic. It emits a PNG signature, an IHDR chunk (8-bit RGBA, no interlace), exactly one IDAT chunk, and IEND. The zlib stream begins with `78 01`, uses *stored* deflate blocks of at most 65,535 bytes with BFINAL set on the last, and ends with an Adler-32 checksum. Every scanline begins with filter byte zero. CRC-32 is computed by hand per chunk. Two encoders that follow these rules produce the same bytes for the same sprite.

One accepted digest. The indexer accepts exactly one v8 mint-body digest per $(\text{seed}, \text{sat}, \text{name}, \text{gen_hint})$: the SHA-256 of `live_v8_html(seed, sat, name, gen_hint)`. The released v7 document and digest are accepted only when the anchor sat equals `GRANDFATHERED_SAT` (Section 4.6). Version 6 bodies (`0x06`) and every other v7 mint are rejected.

The fold source must not declare `var V8` in the same scope as the document's `V8={...}`. The v6.0 hoisting bug is the reason the document assigns the object first and the fold only reads it, never re-declares it.

4.4 The crossing body

The crossing body is exactly 55 bytes.

$(\text{sat}, \text{mint_txid}, \text{mint_index})$ identify the frog and are immutable for its life. `dir` is per crossing: it names the direction the player committed to and therefore the leaf that was spent (Section 6). A body with `dir > 2` is malformed and rejected.

The mint inscription id

```
{reversed_mint_txid}i{mint_index},
```

Table 5: *The `FRG8CROSS` state body. The legacy `FRG7CROSS` spelling is accepted only for the grandfathered sat.*

Offset	Length	Field	Encoding
0	9	magic	`"FRG8CROSS"` (legacy `"FRG7CROSS"`)
9	1	version	`0x08` (legacy `0x07`)
10	8	sat	`u64` little-endian
18	32	mint txid	internal byte order
50	4	mint index	`u32` little-endian
54	1	dir	0 left, 1 straight, 2 right

is the ordinal parent of every crossing inscription. The body stores the same identity in binary, with the txid in internal order. A crossing body therefore binds four things at once: the sat, the mint transaction, the mint output and the chosen direction. A witness copied from another frog cannot be replayed, because the body would not match.

4.5 The fold algorithm

The fold is the canonical state reconstruction. It is stateless with respect to heights: it asks the viewer's ord node for the current tip, walks the crossing children of the mint, verifies each body-valid child against its raw transaction, and replays the children in confirmation order. The children endpoint is followed as `{"ids":[...],"more":bool,"page":n}` until `more` is false. A reorganization is handled by asking again; no local history is authoritative.

For a candidate child c with decoded direction dir, the fold uses three predicates. The first is the body check:

Listing 3: *Body, raw-transaction and proof predicates.*

```
bodyOk(c) = content(c) is exactly 55 bytes,
            starts FRG8CROSS || 0x08 || sat || mint_txid || mint_index,
            has dir <= 2, and the content type is octet-stream

raw(c)    = GET /r/tx/{c.txid}

proofOk(c)= raw(c) is version 2, locktime 0, all input sequences 0xffffffff,
            has exactly one envelope in input 0 and no pointer or extra
            envelope, output 0 has value > 0, the input-0 tapscript is the
            canonical 161-byte leaf for body(c).dir, and its 97-byte control
            block has NUMS as internal key and the canonical sibling path.
            A confirmed Bitcoin transaction is already consensus-valid; the
            fold only has to check that the spent leaf is the canonical one.
```

A body-valid child whose proof is missing or false makes the fold *unverified*; it is never silently skipped. A body-invalid child (wrong magic, wrong sat, wrong mint, malformed direction) is ignored. Incomplete pagination, with `more` still true after the bounded cursor, is also `unverified`, never a lower-bound count.

Three details are consensus-critical. First, a child is admitted only when its sat and satpoint equal the mint's, its body is the canonical 55-byte body for this frog and mint, and its raw transaction proves the canonical state leaf was spent. A parent tag alone is never trusted. Second, the outcome is recomputed from the block hash and the direction in the body: the fold never asks the HOPPER indexer what happened; it verifies the spend from ord's raw transaction endpoint. Third, the finality check rejects a second crossing before κ blocks even when both transactions are already confirmed at the current tip. A transfer remains invisible to `dist`, but it cannot create a false `HOME`: the next crossing lacks a canonical state leaf and fails the proof.

Algorithm 1 The canonical proof-checking fold

Input: sat, own inscription id, and the embedded parameters

id ← own inscription id from `location.pathname`
mint ← parent of id (the mint document is its own mint)
tip ← Get`(/r/blockheight)`
last ← Get`(/r/inscription/{id})`
$C \leftarrow [\,]$
for each page of Get`(/r/children/{id})` until `more` is false **do**
  for each c in the page's `ids` **do**
    $d \leftarrow$ Get`(/r/inscription/{c})`
    if d.sat = sat and d.satpoint = last.satpoint and d.height is set **then**
      if BodyOk(c) **then**
        if not ProofOk(c) **then**
          return unverified("proof")
        end if
        append (c, dir) to C
      end if
    end if
  end for
end for
if More is still true after the bounded cursor **then**
  return unverified("pagination")
end if
sort C by $(\mathrm{height}, \mathrm{sequence_number})$
state ← idle; dist ← 0; lastT ← null; txids ← ∅
for each (c, dir) in C **do**
  if c.id ∈ txids **then**
    return unverified("duplicate crossing")
  end if
  if lastT ≠ null and c.height − lastT < κ **then**
    return unverified("crossing before finality")
  end if
  $T \leftarrow c$.height
  if $T + \kappa >$ tip **then**
    return crossing(dist, dir)
  end if
  if dir = $w(T+1)$ **then**
    return dead(dist)
  end if
  dist ← dist + 1; lastT ← T; txids ← txids ∪ $\{c.\mathrm{id}\}$
  if dist ≥ $r(T)$ **then**
    return home(dist)
  end if
end for
return idle(dist, $r(\mathrm{tip}+1)$)

4.6 The one grandfathered v7 Hopper

Exactly one released Hopper is carried across the v8 cut. Its sat is the global `GRANDFATHERED_SAT` value:

- Rust: `hopper_core::params::GRANDFATHERED_SAT`;
- JavaScript: `GRANDFATHERED_SAT` in `web/js/core/params.js`.

The production value is `1_965_983_893_977_665`; `0` means none. It is the only value to edit when the sat changes, and it is embedded into new v8 mint documents as `grandfatheredSat` so the fold does not need a second copy.

For that sat only:

- the v7 mint envelope (`0x07`), the v7 document digest and the `V7` body are accepted;
- the v7 state body (`FRG7CROSS`/`0x07`) and the v7 161-byte leaf and control path are accepted;
- the wallet uses the released leaf and version and can recover or migrate the sat on the legacy Taproot derivation (Section 8).

Every other sat must use v8. A v7 crossing for a non-grandfathered sat is rejected by the indexer and is not counted by a v8 fold; the app and indexer report it as `unverified`. The grandfathered Hopper is not a precedent for a second legacy frog.

The released v7 card's embedded fold is immutable and predates the proof-checking algorithm. It is grandfathered for custody and play, not as an authoritative art oracle; a viewer that needs the proof-checked state must use the v8 app or indexer, or a new v8 card.

No hidden state

Everything the fold needs is in the mint document, the crossing bodies, the raw transactions, the block hashes and the ord node's satpoint answers. There is no HOPPER database in the trust path, and a body is never believed merely because it is attached to the right parent.

5 Game semantics

5.1 The frog record

The indexer keeps one record per frog. The record is a projection of chain events; it is not an independent source of truth.

Table 6: *The frog record.*

Field	Meaning
`sat`, `seed`, `name`	Identity and display data from the mint
`gen`	$\gamma(\mathrm{mint_height})$, immutable, 1-based
`state`	`idle`, `crossing`, `home`, `dead`, `burned`, or `retired`
`dist`	Completed crossings
`crossings_entered`	Crossings ever accepted, including those in flight
`crossings_settled`	Finished attempts: survived, dead, or burned
`deaths`	0 or 1; a dead frog never crosses again
`mint_height`, `mint_txid`, `mint_index`, `inscription_id`	Mint identity
`wire_version`	`0x08`, or `0x07` for the grandfathered sat
`location`	`tracked(outpoint)` or `burned`
`in_flight`	`{txid, height T, dir, target_hops = r(T)}`
`outcome_block_hash`, `outcome_wrap`, `provisional_dead`	Frozen outcome at $T + 1$
`last_target_hops`	Target of the most recent crossing
`immortal_height`, `death_height`, `burn_height`, `burn_txid`	Terminal history

The states `home`, `dead`, `burned`, and `retired` are absorbing. `retired` is a reserved absorbing state; the transitions described here do not create it. A frog that is `idle` can hop, transfer, or burn; a frog in `crossing` has a direction already committed, an outcome fixed at $T + 1$, and a settlement pending at $T + \kappa$. The generation label `gen` is fixed at mint and never changes, even across a difficulty reset.

5.2 Mint

A transaction is a valid mint if and only if all of the following hold.

1. The block is at or after the frozen activation height `974,034`. There is no lenient production mode.
2. Exactly one inscription reveal exists in the transaction, it is on input 0, its ord pointer is absent, its envelope is valid, and its input inscription count is exactly one.
3. Its content type is `text/html;charset=utf-8`.
4. The body is the canonical `live_v8_html(...)` for `(seed, sat, name, gen_hint)`. For the grandfathered sat only, the canonical `live_v7_html(...)` document, its digest and the v7 mint envelope are also accepted.
5. The first sat of output 0 exists and is not a shadow; the tracker has seen the inputs that created it.
6. That sat carries no frog, live or dead.

A valid mint creates a record in state `idle`, with `dist = 0`, `location = tracked(mint txid, 0)` and `gen` = γ(`mint_height`). The `gen_hint` embedded in the document is ignored. The mint output is *not* a state output. Its script is the wallet's frog address, a BIP86 Taproot output; it does not commit to the mint id, because the mint transaction cannot contain its own txid. This is the reason for the one-time prepare transaction described in Section 6.

5.3 Hop

A transaction is a valid crossing (a hop) if and only if all of the following hold.

1. The transaction contains exactly one inscription, in input 0, with no ord pointer and an input inscription count of exactly one.

2. Its body decodes as a 55-byte body with magic `FRG8CROSS` and version `0x08`, with $\mathrm{dir} \leq 2$, and its $(\mathrm{sat}, \mathrm{mint_txid}, \mathrm{mint_index})$ equals the frog's identity. A legacy body with magic `FRG7CROSS` and version `0x07` is accepted only when the frog is the grandfathered sat and its recorded `wire_version` is `0x07`.
3. The tag-3 parent equals $(\mathrm{mint_txid}, \mathrm{mint_index})$.
4. The spent leaf is the `dir` leaf of the state output's tree: the script is the 161-byte template of Section 6 whose body's last byte is `dir`, its hash is the matching leaf hash, and the 97-byte control block commits the output key of the state output (NUMS internal key, `0xc0`/`0xc1` parity, both siblings in the order of the table in Section 6).
5. Input 0 spends the frog's current outpoint.
6. The anchor sat lands in output 0 at offset 0.
7. No funding input carries a frog, and no `FRG` envelope appears in the crossing.
8. The frog is `idle`: not crossing, home, dead, burned, or retired.

On acceptance the record becomes `state = crossing`, with `in_flight = {txid, T, dir, r(T)}`. The counter `crossings_entered` is incremented, `last_target_hops` is set to $r(T)$, and any previous `outcome_block_hash`, `outcome_wrap` or `provisional_dead` is cleared.

The target is frozen at confirmation and never recomputed. This is what makes the game fair across a resetting schedule: a frog that hops at height T is judged by the road as it existed at T, even if a generation boundary resets the road while the crossing is in flight. Choosing a direction selects a leaf; it cannot change the leaf's bytes, which were fixed when the state output was created.

5.4 Settlement timers

Timers run before a block's transactions. This ordering matters: a crossing that confirms in block T cannot settle in the same block, and the outcome block's hash is known before any settlement at $T+\kappa$ is evaluated.

At block $B = T + 1$, for every frog whose in-flight height plus one equals B,

$$w \leftarrow \mathrm{LE}_{32}\left(H_B^{\mathrm{int}}[0..4]\right) \bmod 3, \tag{9}$$

$$\mathrm{outcome_block_hash} \leftarrow H_B, \tag{10}$$

$$\mathrm{outcome_wrap} \leftarrow w, \tag{11}$$

$$\mathrm{provisional_dead} \leftarrow (\mathrm{in_flight.dir} = w). \tag{12}$$

The value w is computed once per block and applied to every frog in it: the road picks one direction for the block, not one per Hopper.

At block $T + \kappa$, for every due crossing,

Listing 4: *Settlement at $T + \kappa$.*

```
if location is burned or provisional_dead:
    state = (burned ? burned : dead)
    deaths += 1
    death_height = T
else:
    dist += 1
    state = (dist >= target ? home : idle)
    if state == home:
        immortal_height = T
crossings_settled += 1
in_flight = None
```

```
provisional_dead = None
emit Settle event
```

Notice that a burned in-flight frog still settles as `burned` at finality; it is not converted to `dead`. Notice also that `crossings_settled` counts every settled crossing, including a death. The distinction between `dist` and `crossings_settled` is deliberate: `dist` counts survived hops, while `crossings_settled` counts finished attempts.

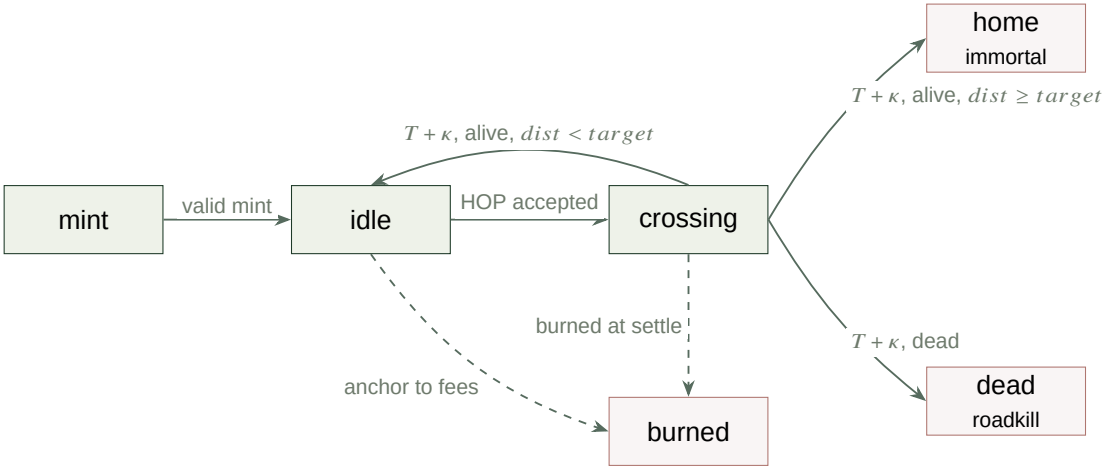


Figure 2: *The v8 frog lifecycle.* `home`*,* `dead`*, and* `burned` *are absorbing. The dashed transitions are terminal or exceptional: an idle frog whose anchor sat is paid to fees is burned immediately, while an in-flight frog that is burned settles as burned at finality.*

5.5 Transfer and burn

A spend of the frog's outpoint without a valid crossing envelope is a transfer: the indexer updates `location` and leaves the state unchanged. No inscription is published, and no state changes. This is the simplest way to move a frog, and it is why a transfer is invisible to the fold.

A spend that sends the anchor sat to fees marks `location = burned` and sets `burn_height` and `burn_txid`. An idle frog becomes `burned` immediately, because nothing else will observe the sat again. An in-flight crossing keeps its timer and settles as `burned` at finality. A wallet should not build, offer, or expose burn as a feature or operation; the indexer still classifies such a spend if it is built elsewhere.

The asymmetry is intentional. A transfer of an idle frog is a normal custody move. A burn is irreversible, and the record must say so at the moment the sat leaves the tracked set. For an in-flight frog, the outcome at $T + 1$ has already been frozen; destroying the sat does not undo the crossing, it only changes how the settlement is labelled.

5.6 Events

The indexer publishes a re-derivable log. Each entry is keyed by $(\text{block_hash}, \text{height}, \text{txid}, \text{action})$ and has one of four shapes.

Table 7: *Event payloads.*

Action	Payload
`mint`	`{frogger_sat, seed, name, gen}`
`transfer`	`{frogger_sat}`
`settle`	`{frogger_sat, outcome: dead|crossed, dir, wrap}`
`burn`	`{frogger_sat}`

The log is not a second consensus. It is a projection of the same events the state machine applied, and it can be regenerated from the chain. `outcome: crossed` means the Hopper survived that hop, not that it reached `HOME`. `HOME` is the frog record's `state = home` with `immortal_height = T`; the road log reads the frog record, not the bare event. The `burn` action remains an indexer/API state, not a wallet feature.

5.7 Reorganizations

The indexer keeps a retained window of block headers and undo journals. A rollback replays the chain to the fork point. Timers are pure functions of height and hash, so a reorg that replaces $T + 1$ freezes a new outcome, and a reorg that removes T reopens the frog as `idle` and restores its card. No manual repair is required.

Settlement is monotone before finality

Before $T + \kappa$, the only committed fact is the crossing itself. The outcome hash, the wrap and the provisional verdict may be replaced by a reorg. At $T + \kappa$, the retained window is what allows the indexer to replay a fork without losing the ability to settle.

5.8 A crossing in six acts

The whole protocol can be read as six steps.

1. The wallet derives the state output for $(\mathrm{sat}, \mathrm{mint_id}, \mathrm{owner})$.
2. The player chooses left, straight or right. The wallet signs the matching cross leaf with the owner key and includes funding and change as needed.
3. It broadcasts one transaction. The transaction carries exactly one inscription, in input 0, and sends the anchor sat to output 0 at offset 0.
4. The indexer admits the crossing at height T, freezes $r(T)$, records the chosen direction, and emits a `crossing` state. The art fold reports crossing.
5. At $T + 1$ the outcome hash is recorded and the block's wrap is computed. No settlement happens yet.
6. At $T + \kappa$ the timers run: `home` if the distance reached the target, `idle` if the frog survived but has more to go, or `dead` if its chosen direction equals the block's wrap.

6 Custody: the Taproot tree that plays the game

6.1 The state output

A frog in custody is a Taproot output with the fixed internal key `NUMS_XONLY`:

`50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0`.

No discrete log of this point is known, so there is no key-path spend. The output can only be spent through one of four leaves. The tree is balanced at depth two: three crossing leaves L, S, R (left, straight, right) and one send leaf X (transfer).

$$
\begin{aligned}
h_L &= \mathrm{leaf_hash}(L), & h_S &= \mathrm{leaf_hash}(S),\\
h_R &= \mathrm{leaf_hash}(R), & h_X &= \mathrm{leaf_hash}(X),\\
h_{LS} &= \mathrm{branch}(h_L, h_S), & h_{RX} &= \mathrm{branch}(h_R, h_X),\\
\mathrm{root} &= \mathrm{branch}(h_{LS}, h_{RX}), & Q &= \mathrm{taproot_output_key}(\mathrm{NUMS}, \mathrm{root}),
\end{aligned}
$$

where

$$\mathrm{branch}(a, b) = \mathrm{tagged_hash}(\text{“TapBranch”}, \min(a, b) \,\|\, \max(a, b)), \tag{13}$$

and

$$\mathrm{leaf_hash}(s) = \mathrm{tagged_hash}(\text{“TapLeaf”}, \texttt{0xc0} \,\|\, \mathrm{compact_size}(|s|) \,\|\, s). \tag{14}$$

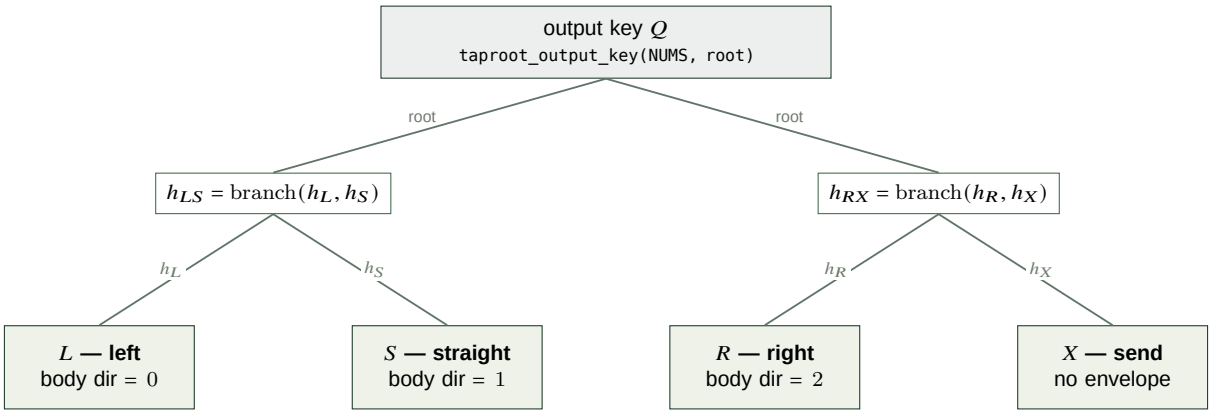


Figure 3: *The four-leaf state output. The internal key is a nothing-up-my-sleeve point; all leaves sit at depth two. L, S and R are the crossing leaves, identical except for the direction byte in the body they push. X is the send leaf. There is no key-path spend.*

Every control block is 97 bytes:

$$\begin{aligned}&[\mathrm{parity} \mid \texttt{0xc0}] \,\|\, \texttt{NUMS_XONLY} \\ &\quad \| \,\langle\text{sibling leaf hash}\rangle \,\|\, \langle\text{sibling subtree hash}\rangle.\end{aligned} \tag{15}$$

Table 8: *Control-block siblings by spent leaf.*

Spent leaf	First sibling	Second sibling
L (left)	h_S	h_{RX}
S (straight)	h_L	h_{RX}
R (right)	h_X	h_{LS}
X (send)	h_R	h_{LS}

The three crossing leaves differ only in the direction byte of the body they push (Section 4). Each leaf hash is therefore fixed when the state output is created. Choosing a direction selects a leaf; it cannot change the leaf’s bytes. The current tree uses the `FRG8CROSS/0x08` body. A state output whose sat equals `GRANDFATHERED_SAT` may use the released `FRG7CROSS/0x07` body and its own leaf hashes; the two versions are never mixed in one tree.

`owner_xonly` is the 32-byte witness program of a BIP86 P2TR address derived at

$$m/84'/\mathrm{coin}'/0'/2/0, \tag{16}$$

the ordinals branch. Its BIP86 even-Y normalized output is the wallet’s current frog receive address and the state-leaf owner. The released implementation tweaked the raw point without normalizing; a recovery scan also watches that legacy address and derives the legacy state scripts. The sender parses the recipient’s P2TR address to obtain the key; a non-P2TR destination is refused for custody.

6.2 The cross leaf

All three crossing leaves are the same script; only the last byte of the pushed body (the direction) differs:

Listing 5: *Cross leaf template.*

```
20 <owner:32>          ; PUSH32 owner
ad                     ; OP_CHECKSIGVERIFY
00                     ; OP_FALSE
63                     ; OP_IF
03 "ord"               ; PUSH 3 "ord"
01 01                  ; push tag 1
18 <content-type:24>   ; PUSH 24 "application/octet-stream"
01 03                  ; push tag 3
20 <parent:32>         ; PUSH32 mint txid (index 0)
00                     ; empty body tag
37 <body:55>           ; PUSH 55 bytes, last byte = dir; FRG8CROSS/0x08
68                     ; OP_ENDIF
51                     ; OP_TRUE
```

The first part is an ordinary `CHECKSIGVERIFY` against the owner key. The second part is an ord envelope that is never executed as a condition: `OP_FALSE` skips the `OP_IF` branch, so the envelope is data, not control flow. The final `OP_TRUE` leaves a true stack value after the signature check.

Every push is minimal. The content type is the 24-byte string `application/octet-stream`, pushed with `0x18`. The parent tag is the 32-byte mint txid for index 0 — the only reachable case, since a valid mint carries exactly one envelope and it sits on input 0 — or the txid plus the minimal little-endian index (a push of length 33 to 36 bytes, opcode `0x21` to `0x24`). The script is therefore **161 bytes** at index 0, and up to 165 bytes for a manually built nonzero index. The three direction leaves are the same length; only the body's last byte differs.

The send leaf is deliberately smaller:

Listing 6: *Send leaf.*

```
20 <owner:32> ac
```

It is 34 bytes and contains no envelope. A transfer is therefore silent: it moves the sat but publishes no state.

6.3 Control blocks and witnesses

A custody spend has three witness elements:

Table 9: *Witness stacks for the two custody spends.*

Spend	`witness[0]`	`witness[1]` / `witness[2]`
hop	64-byte BIP-340 [4] signature ‖ `0x01` (SIGHASH_ALL)	the chosen `dir` leaf script / 97-byte control block
transfer	64-byte signature ‖ `0x01`	send leaf script / 97-byte control block

SPEC.md specifies that the signature is produced with the tweaked owner key over the BIP-342 tapscript sighash of the leaf and the transaction's whole prevouts and outputs. The reference implementation expresses that key as `internal_key.tap_tweak(None)`. The SIGHASH_ALL byte commits every input and output, so a signature cannot be redirected to a different transaction.

6.4 The two-phase mint

Ord needs an input to engrave an inscription, so a mint is a commit/reveal pair. The commit output is a Taproot output with the BIP86 owner key (`owner_xonly`) as internal key and one leaf:

Listing 7: *Commit output leaf.*

```
<owner:32> OP_CHECKSIG OP_FALSE OP_IF <mint inscription envelope> OP_ENDIF
```

Its address is $\mathrm{p2tr}(\mathrm{owner}, \mathrm{merkle_root}(\mathrm{leaf}))$, and its value is

$$1{,}000 + \mathrm{reveal_fee_budget}(\mathrm{feerates}). \tag{17}$$

The leaf demands the owner's signature. SIGHASH_DEFAULT commits every prevout and every output, so a copied witness cannot redirect the frog. A legacy keyless commit, using `NUMS` as internal key with `OP_TRUE || envelope`, remains revealable forever, but no new commit may use it.

The reveal transaction then has the shape in Table 10.

Table 10: *The mint reveal transaction.*

Position	Role	Content
input 0	commit spend	script path `[sig, leaf, control block]`
output 0	frog	1,000 sats to the wallet's frog P2TR key-path address
output 1	mint message	OP_RETURN `FRG 0x08 0x01 seed name` (`0x07` only for the grandfather)
output 2	change	`m/84'/coin'/0'/1/index`, optional if ≥ dust

Every input uses sequence `0xFFFFFFFF`, so the transaction is not replaceable. The anchor sat is the first sat of the commit output, which is the first sat of the first funding input. Because the commit leaf embeds the anchor sat, the commit address is derived after a probe coin-selection pass.

6.5 Action transactions

After the mint, a frog moves through the transactions in Table 11. All of them use version 2, lock time zero, and sequence `0xFFFFFFFF` on every input. They are intentionally not replaceable: a mint, prepare, hop or transfer may move an inscription or a state card, and shaving an output to raise a fee can burn the anchor sat.

Table 11: *The custody actions.*

Action	Input 0	Outputs
prepare	the frog output, P2TR key path with the tweaked frog key	output 0 is the state output for $(\mathrm{sat}, \mathrm{mint}, \mathrm{owner})$; outputs 1.. are funding change
hop	the state output, the leaf for the chosen direction	output 0 is the next state output (same owner or a new P2TR owner); outputs 1.. are funding change
transfer	the state output via the send leaf, or the mint output via key path	output 0 is the recipient's ordinary address (any script, P2WPKH included)

One crossing per transaction

A hop publishes exactly one inscription, in input 0. The anchor sat must land in output 0 at offset 0. No funding input may carry a frog, and no `FRG` envelope may appear anywhere in the crossing.

prepare exists only for a frog still in its mint or frog-address output. The state leaf commits to the mint inscription id, which is created by the mint reveal; an output of that transaction cannot contain its own txid. The first state output is therefore created by either the first transfer or the one-time prepare. After that, each crossing spends the previous state output and creates the next one.

The player chooses $\mathrm{dir} \in \{\mathrm{left}, \mathrm{straight}, \mathrm{right}\}$ before signing; the transaction spends the matching leaf, so the body it inscribes carries that `dir`. Choosing a direction is the whole of the choice: the leaf's bytes are fixed by the state output, and the outcome is decided by the block, not by the player. The wallet must show the chosen direction in the preview and must not let it change after the preview is approved.

A transfer deliberately pays the recipient's ordinary address, not a state output. The state script commits to the mint id, which the recipient cannot know until it has discovered the frog. The recipient re-enters custody with prepare on its next action.

6.6 Previews and the ordinary send

A browser send (including **SEND ALL**), hop, transfer or mint is previewed before broadcast. send and transfer display `PREVIEW TRANSACTION`; **SEND ALL** is titled `SEND ALL BITCOIN`; mint keeps its `MINT!` button and opens the same review window; a hop is previewed with its direction spelled out (`HOP LEFT`, `HOP STRAIGHT` or `HOP RIGHT`) and the leaf it will spend. The preview shows the destination address and amount, change address and amount (or "one output" for a transfer or **SEND ALL**), chosen fee rate and actual fee in sats. A mint shows the commit and reveal txids, both change outputs, both fees and the total size.

`CANCEL` performs no broadcast, persists no action and spends nothing. Approval re-scans and refuses the preview if any input moved, an anchor's ordinal ranges changed, or a coin gained a Hopper, then broadcasts the exact previewed bytes. A mint approval broadcasts the commit and then the reveal that spends it; if the reveal is rejected after the commit was accepted, the error names the accepted commit and no mint action is recorded. A mint never has an RBF path. Fee recommendations come from mempool.space `/api/v1/fees/precise` (`fastestFee`, `halfHourFee`, `economyFee`); the chosen rate remains manually editable, and values below the 1 sat/vB wallet relay floor are raised to it.

An ordinary wallet send is separate from these actions. Its inputs use sequence `0xFFFFFFFD` (BIP125 opt-in), and the wallet may replace an unconfirmed send only under a narrow policy: the original is a wallet-created plain BIP84 send; the backend returns exactly the recorded serialization; every original input is a plain, range-known, non-Hopper BIP84 coin and the current view still contains no Hopper at those outpoints or sats; every original output keeps its exact script and value; the replacement appends confirmed plain BIP84 funding and one new change output; and it pays a strictly higher absolute fee and feerate, including BIP125's incremental relay margin. The wallet never replaces a HOPPER action. The transaction builder recomputes the FIFO sat ranges and refuses a replacement if a known Hopper sat would land in its fee.

SEND ALL. A SEND variant sweeps the plain-BTC balance into exactly one recipient output. It selects every eligible plain BIP84 coin: complete, value-matched ranges, no Hopper or state output, and no protected sat; unconfirmed coins are included only when the wallet permits them. There is no change output, and the actual fee is deducted from the total, so the recipient receives $\mathrm{totalIn} - \mathrm{actualFee}$. The build refuses when the fee consumes the balance or leaves the recipient below the P2WPKH dust limit, and approval repeats the same exact-bytes and input checks as SEND. The preview is titled `SEND ALL BITCOIN` and states that there is one output and no change.

6.7 Sat assignment

Ordinals move FIFO over inputs in order. The protocol depends on three consequences.

1. The anchor sat is the first sat of input 0. For a mint, that is the first sat of the commit output.
2. After a spend, the anchor sat is at offset 0 of output 0 if and only if output 0 receives value and no earlier output claims sats first.
3. A fee is taken from the tail of the frog's range or from funding; it is never taken from the anchor's position.

The wallet and the indexer both verify these properties with ordinal range arithmetic. A coin whose ranges cannot be attributed is not spent.

6.8 Addresses, discovery and history

- The mint pays the wallet's **frog address**: the BIP86 P2TR output at `.../2/0`. It never pays a BIP-84 receive address. Coins at the frog address are never funding candidates.
- The BIP-84 receive script is **fixed at `.../0/0`** in v8. This is a deliberate simplicity and recovery tradeoff, not an accidental gap: the watch set is fixed, paginated to exhaustion, and a restored wallet reuses the same script. Address rotation and privacy are future wallet features, not a consensus rule. The UI must not claim that a receive address retires.
- The state output's script is per $(\mathrm{sat}, \mathrm{mint\ id}, \mathrm{owner})$. It matches no derivation path. A wallet finds it by deriving the script from the frog record and by watching its frog address.
- A frog sent by a peer arrives at the ordinary address and is discovered by the wallet's address scan plus the indexer or ord. It is imported by reading its mint inscription and replaying its history from the mint outpoint.
- History is wallet-scoped, not a raw indexer feed. It merges local actions, events whose Hopper sat belongs to this wallet, settled outputs observed on its watched scripts, internal change, and mempool rows. A confirmed ordinary deposit or a Hopper output is retained as a `receive` row, so it stays visible after the coin is spent. Change from our own transactions is retained separately as a `CHANGE` row; it is never presented as an incoming `RECEIVE`. Events for unrelated public or NPC Hoppers are filtered out.
- A `MINT` row does not depend only on the browser that broadcast it. When the owned mint output is still at its mint outpoint, the wallet persists a chain-derived mint row, so a restored seed shows the same `MINT` and `HOPPER ARRIVED` rows without a local action ledger. A Hopper received from a peer is not at its mint outpoint and does not create a `MINT` row.
- Rows for one txid share its best-known height and sort as a group. The feed is newest first, so inside a group the newest row is on top: `HOPPER ARRIVED` → `CHANGE` → `MINT`, and `CHANGE` → `SEND`. A mint's commit change is grouped with the reveal's `HOPPER ARRIVED` row by the minted sat and by both txids. Older actions with no stored sat recover it from the arrival's frog/reveal record; a missing commit is read once from input 0 of the reveal and persisted. A missing sat is never treated as sat 0. Grouping is ultimately by reveal/commit txid cluster, so it works even when no row stores a sat. Shortened txids and addresses are copy buttons: the visible text is abbreviated, but the copied text is the complete value.
- The first completed refresh after the history-ledger upgrade performs two one-time, versioned address-history backfills: receive addresses the wallet handed out, and change addresses it has used. The receive pass recovers deposits whose coins were spent before the receive ledger existed and skips outgoing action/commit txids; the change pass recovers spent internal change and migrates older ledgers that had stored it as a `RECEIVE`. `RESCAN` clears both markers so recovery can be forced.

No key-path escape hatch

A state output is a UTXO spent exactly once. The body binds the sat, the mint and the direction, so a witness cannot be replayed against another frog. The NUMS internal key ensures that the only ways out are the four leaves.

7 The standalone indexer

7.1 Crate boundaries

The Rust implementation is a three-crate workspace.

Table 12: *Indexer crates and responsibilities.*

Crate	Responsibility
`hopper-core`	Pure protocol: envelope, v8 and legacy body, scripts, parameters, generations, wrap, sat math, art, PNG, fold document. Depends only on `sha2` (plus optional serde).
`hopper-indexer`	Pure state machine: sat tracker, rules, timers, events, snapshots, optional redb store. Performs no network I/O; the optional redb store is local disk persistence.
`hopper-indexer-api`	Axum HTTP API and block ingest (ZMQ/RPC or admin POST). Owns the network and the write lock.

The separation is not cosmetic. `hopper-core` can be compiled for native and mobile targets without a Bitcoin node. `hopper-indexer` can be replayed from a fixture without sockets. Only the API crate knows about ZMQ, RPC, HTTP, or authentication.

7.2 Ingest

The indexer always ingests from block 0. A later start cannot name coins created earlier; a frog whose mint predates the start would look as if it had been sent away. This is a correctness rule, not an optimization.

There are two wires, sharing the same block decoder and the same write lock.

- `POST /v1/admin/block` accepts an operator or debug JSON block. Outside loopback it requires the admin token.
- bitcoind ZMQ `hashblock` plus RPC is the production path. On a new hash, the indexer fetches blocks from its tip to that hash. On a reorg, it rolls back to the fork and replays.

The ingest layer converts a Bitcoin transaction into a small, witness-light structure:

Listing 8: *Ingest types.*

```
Tx {
    txid,
    inputs[{ prevout, value, inscription? }],
    outputs[{ value, op_return? }]
}
Block { hash, height, txs }
```

It parses the tapscript witness at `witness[n-2]` with the ord envelope decoder, extracts the control block at `witness[n-1]`, and records the content type, body and tag-3 parents. The state machine never touches witnesses.

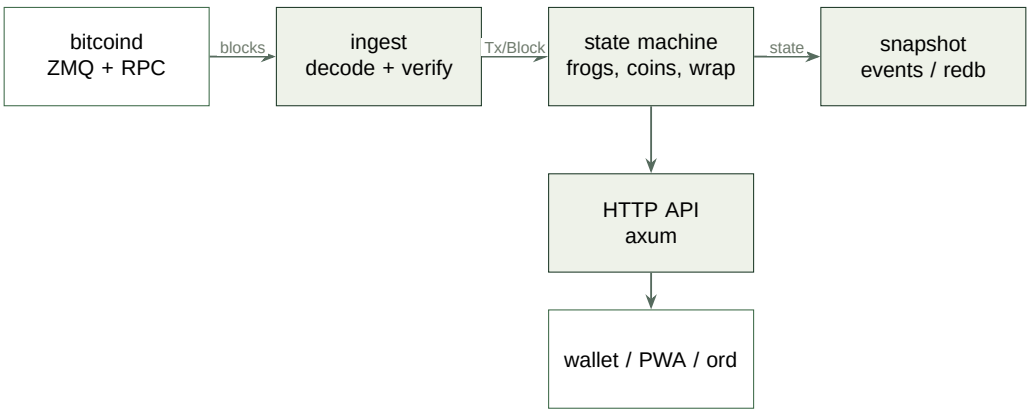


Figure 4: *Indexer architecture. The state machine is independent of the network and of the HTTP layer; persistence is a projection of the same state.*

7.3 State, snapshots and the state root

The state is a function

$$(\text{config}, \text{seeds}, \text{chain}) \longrightarrow \text{IndexState} + \text{event log}. \tag{18}$$

It contains the frog records, the coin ranges, the retained headers, the pending timers and the tip. A canonical state root hashes the parameter commitment, the tip (height and hash), the ordered frog records and the coin ranges. Two indexers that agree on consensus must agree on the root; a disagreement is a bug or a different parameter set, not a client preference.

Persistence has three layers.

- A binary snapshot at `HOPPER_SNAPSHOT_PATH`, default `hopper-indexer.snapshot.bin`. The snapshot embeds the parameter hash; a mismatch on load is a hard error, never a migration.
- Append-only event segments next to the snapshot.
- An optional redb durable store (feature `redb`, path `HOPPER_DB`) holding the full state and the undo journals.

The snapshot interval defaults to 10 blocks, the retained block window to 288 blocks, and the event segment size to 4096 events.

7.4 HTTP API

The public surface is read-only. Administrative routes are separate and require a Bearer token when `HOPPER_ADMIN_TOKEN` is set; the server refuses a non-loopback bind without one. They are never exposed by the PWA.

Table 13: *Indexer HTTP API.*

Route	Purpose
`GET /v1/status`	Tip, height, active crossings, wire version
`GET /v1/params`	Full parameter block, `params_hash`, chain id
`GET /v1/generation/{height}`	Generation index, start, end, `required(h)`
`GET /v1/wrap/{height}`	The block's wrap ($0\|1\|2$); 404 until ingested
`GET /v1/froggers`	Paginated list; `limit`, `offset`, `after_sat`, `to_sat`
`GET /v1/frogger/{sat}`	One frog record, with `gen`, `in_flight.dir` and `outcome.wrap`
`GET /v1/road`	In-flight crossings: T, direction, target, wrap, settle height
`GET /v1/leaderboard`	Ranked by generation, then `dist`; `limit`
`GET /v1/state/root`	Canonical state root
`GET /v1/events`	Event log; `limit`
`GET /v1/events/stream`	Server-sent event stream
`GET /v1/mempool`	Mempool preview; needs RPC
`GET /v1/fees`	bitcoind fee estimate; never invented
`GET /v1/utxo/{txid}/{vout}`	Value, ordinal ranges, frogs, `exact` flag
`GET /healthz`, `/readyz`, `/metrics`	Liveness, readiness, metrics
`POST /v1/admin/block`	Ingest an operator/debug block
`POST /v1/admin/seed`	Bootstrap explicit ordinal ranges
`POST /v1/admin/rollback`	Roll back to a fork point

CORS is a browser read policy configured by `HOPPER_CORS_ORIGINS`. It is not access control and must not be treated as one. The API never publishes a wrap for a height it has not ingested: the outcome of a future block is not predictable from the indexer either.

7.5 Configuration

Table 14: *Indexer configuration. Legacy `FROGGER_*` names are accepted as fallbacks.*

Variable	Meaning / default
`HOPPER_API_ADDR`	Bind address; default `127.0.0.1:9770`
`HOPPER_ACTIVATION_HEIGHT`	Frozen inscription cutover; default and required `974,034`
`HOPPER_INSCRIPTION_ACTIVATION_HEIGHT`	Frozen cutover; default and required `974,034`
`HOPPER_RATE_LIMIT_PER_MIN`	Public-read budget per caller; default 300, `0` disables
`HOPPER_CORS_ORIGINS`	Browser read origins; default *; not access control
`HOPPER_SNAPSHOT_PATH`	Snapshot file; default `hopper-indexer.snapshot.bin`
`HOPPER_SNAPSHOT_EVERY`	Snapshot interval; default 10 blocks
`HOPPER_RETAIN_BLOCKS`	Reorg window; default 288
`HOPPER_EVENT_SEGMENT`	Events per append-only segment; default 4096
`HOPPER_EVENT_RETAIN_SEGMENTS`	Old segments retained; default 0 (keep all)
`HOPPER_SEEDS_FILE`	Bootstrap seed JSON for fresh starts
`HOPPER_INGEST_START_HEIGHT`	Must be 0 or unset; later starts are refused
`HOPPER_ORD_URL`	Optional ord base for lazy range seeding
`HOPPER_ZMQ_ADDR`	bitcoind `zmqpubhashblock`, e.g. `tcp://127.0.0.1:28332`
`HOPPER_RPC_URL`	bitcoind RPC; default `http://127.0.0.1:18443`
`HOPPER_RPC_COOKIE`	RPC authentication
`HOPPER_RPC_USER`	
`HOPPER_RPC_PASS`	
`HOPPER_NETWORK` / `HOPPER_GENESIS_HASH`	Published chain identity
`HOPPER_ADMIN_TOKEN`	Bearer token for `/v1/admin/*`; a non-loopback bind without it is refused
`HOPPER_DB`	Optional redb path
`HOPPER_DB_CACHE_MIB`	redb page cache in MiB; 0/unset uses redb's default
`HOPPER_ENGINE=store`	Keep only a block-sized overlay in memory
`HOPPER_REINDEX=1`	Discard the durable store

The legacy `FROGGER_*` spellings are read as fallbacks so an existing deployment can migrate without editing its service file.

7.6 Seeding

Ordinal ranges ultimately come from the chain. A fresh indexer ingests from genesis, but a deployment that starts from a snapshot, or that is handed a UTXO it never saw move, can be given explicit ranges:

Listing 9: *Admin seed request.*

```
POST /v1/admin/seed
{
  "utxos": [
    { "txid": "<display hex>", "vout": 0,
      "ranges": [[start, end]] }
  ]
}
```

A coin whose ranges were never observed is a shadow sat. A mint on one is rejected as `MintAnchorUntracked`. Seeding is an operator bring-up step, not a consensus rule.

7.7 Rejections

No malformed input is chain-invalidating. A rejection leaves state unchanged while sat movement still applies. The closed set lives in `hopper-indexer/src/types.rs` as `RejectionReason`; its families are:

- mint anchor issues (untracked, shadow, already carrying a frog);
- inscription content type and mint-body digest issues;
- crossing body, parent, leaf or control-block issues;
- the direction set: `dir > 2`, the spent leaf is not the `dir` leaf, or the control block is not the matching row of Table 8;
- crossing outpoint and output-position issues;
- crossing on a non-idle frog;
- funding inputs carrying a frog;
- mixed `FRG` envelope;
- parameter-hash mismatch.

A rejected transaction is not a reorg and not a chain failure. It is a protocol message that the state machine declined to apply. The API also refuses to serve a mismatched parameter set: `GET /v1/params` publishes the hash, wire version and chain identity, and the wallet refuses to sign without a matching `network`. Missing identity is not permission.

Deterministic replay

Given the same blocks, the same seeds and the same parameter set, the indexer produces the same state root and the same event log. Network topology, arrival order and wall-clock time are not inputs.

8 The wallet and the PWA

8.1 Keys and the vault

A wallet is a 12-word BIP-39 mnemonic. The full 64-byte BIP-39 seed is derived through BIP-32 to

$$m/84'/\mathrm{coin}'/0', \tag{19}$$

with coin type 0 on mainnet and 1 elsewhere. The receive script is fixed at $m/84'/\mathrm{coin}'/0'/0/0$ and change addresses are $m/84'/\mathrm{coin}'/0'/1/i$; both are P2WPKH. The fixed receive script is a deliberate v8 simplicity and recovery tradeoff. The watch set is fixed, paginated to exhaustion, and a restored wallet reuses the same script; address rotation is a future feature, not a consensus rule.

The ordinals branch is different. The branch is derived at

$$m/84'/\mathrm{coin}'/0'/2/0. \tag{20}$$

The branch point is normalized to even Y and tweaked once with an empty merkle root (the BIP86 [7] tweak). The resulting output key is the 32-byte `owner`: it is the owner in the mint commit leaf and the commit output's internal key, the owner committed by the four state leaves, the key that signs crossings and transfers, and the wallet's current frog receive address. The untweaked branch key is not itself an owner. The released implementation tweaked the raw point without normalizing; a recovery scan also watches that legacy address and derives the legacy state scripts. Migration is explicit: a `migrateLegacyFrog` transfer moves the sat to the current address. The old path is never rewritten in place. This key is not used for funding. Coins at the frog address are never candidates for coin selection, because spending one as fee would burn a frog.

The vault stores the BIP-39 entropy sealed with AES-256-GCM under a non-extractable WebCrypto key-encryption key held in IndexedDB. Creation writes, reads back and verifies the ciphertext before reporting success; a storage denial is an error, never a "sealed" vault. The public ledger (addresses, UTXOs, frog records, history) is stored separately. The ledger is re-derivable from the chain; the 12 words are the only backup.

Table 15: *Wallet derivation paths.*

Path	Script	Purpose
$m/84'/\text{coin}'/0'/0/0$	P2WPKH	Receive (fixed in v8)
$m/84'/\text{coin}'/0'/1/i$	P2WPKH	Change
$m/84'/\text{coin}'/0'/2/0$	P2TR	Frog receive and state owner

8.2 Chain access

The wallet has two HTTP ports, chosen by URL.

Esplora. For an `https://` endpoint, the wallet uses the Esplora API [10]: `/blocks/tip/height`, `/address/{a}/utxo`, `/address/{a}/txs`, `/address/{a}/txs/chain[/{after_txid}]`, `/tx/{txid}/hex`, `/blocks/tip/hash`, `/block-height/{h}`, `/fee-estimates`, and `POST /tx`. Address history is paginated to exhaustion; the first 50 rows are never treated as complete.

Electrum over WebSocket. For a `ws://` or `wss://` endpoint, the wallet speaks JSON-RPC 2.0 and uses `blockchain.scripthash.*`, `blockchain.transaction.get`, `blockchain.block.header(s)`, `blockchain.estimatefee`, and `blockchain.transaction.broadcast`. A relay (`frogger-relay`) can bridge raw TCP.

Discovery comes from the HOPPER indexer (`/v1/froggers`, `/v1/frogger/{sat}`, `/v1/utxo/...`) or, without an indexer, from stock ord (`/r/utxo/...`, `/content/{id}`, `/r/blockheight`). Ranges are mandatory before a coin may be spent. An unattributable coin still counts in the balance, but it is displayed as unspendable rather than spent blindly.

Tip catch-up. Address scans are gated on the tip $(\text{height}, \text{hash})$: outside the catch-up window below, an unchanged tip with no wallet transaction in flight costs no address queries. The tip endpoint can publish a block before the address index has that block's transactions. After an automatic pass that observes a new tip, the wallet must therefore run two further full passes at `TIP_RECHECK_MS` (about 1.2 s) even though the tip is unchanged. Those passes bypass the gate and read the addresses again; only then may the gate skip them. Without this grace window, the first pass could record the new tip while holding the old UTXO set and the gate would keep the stale view until the next block or a forced `RESCAN`. A manual `RESCAN` is a forced pass; it does not itself create this automatic grace window.

Scan repaint. A completed scan is also a UI repaint event. If it lands while a foreground action holds the busy overlay, the repaint must be queued and applied when that action releases the UI, not dropped. Otherwise the Road or Pond can stay on the previous snapshot until the next block or a manual `RESCAN` even though the wallet state is current.

A mint reveal is also self-describing: its tapscript witness contains the canonical v8 body (or the v7 body for the grandfather), and output 0 pays the wallet's ordinals address. A wallet may recover an idle mint record from that witness when neither an indexer nor ord answers, verifying the seed, sat and full body before trusting it. This recovers mints paid directly to the wallet's ordinals address. It does not recover a

frog transferred in later, whose new output carries no mint body; that discovery requires the indexer, or a working ord range view and a record source.

8.3 Screens and interaction

The interface is a single-page PWA with the following screens:

Wallet	History	The Pond	The Road
Mint	Leaderboard	Detail	Bio

These are supplemented by onboarding (choose, show, confirm, recover, mistake), the intro, an eight-page rules walkthrough, the ≡ drawer, send and receive sheets, the address book, backup words, a QR scanner, the dead-frog page, and settings.

The visual language is fixed: dark lily-pad green `#18241a`, card `#223226`, accent `#f0c83c`, 8-bit monospace, square corners, bark-framed buttons, lily-pad-bordered floating windows, and 16×16 deterministic frog sprites. The art is generated from the seed, not fetched from a server.

The road screen asks one question: **LEFT**, **STRAIGHT** or **RIGHT**. The chosen button is the direction committed in the crossing body and the leaf spent; the preview names it, and it cannot change after approval. The screen states the rule in one line: the next block picks one direction, and a Hopper that picked it dies. A frog's card shows its generation badge (`G1`, `G2`, …), its crossing count beside the road mark, and `RIP` before the name once it is dead. Each settled hop gets one road-log row. A crossed row says **survived**; it says **came home** only when that settle's `immortal_height` is the crossing height for the row ($\mathrm{row.height} - \kappa$). A dead row says **died**. If the frog record is absent, for example after a transfer out of this wallet, the row is conservative and says **survived** rather than claiming `HOME`.



Figure 5: *Deterministic 16×16 frog sprites. The same seed produces the same sprite in the Rust implementation, the JavaScript wallet and the inscribed fold.*

8.4 Sound

The score is one MIDI file (`music.mid`) played by `sound.js`. The player parses the file once, maps ticks to seconds through the tempo map, schedules notes with a 400 ms lookahead, and loops at the score's length. Each note is one periodic-wave voice with a harmonic stack falling as $1/k^{1.15}$, a 4 ms attack, and a pitch-dependent decay. The signal chain is a 45 Hz highpass, a +2.5 dB shelf at 1.5 kHz, a compressor

at -16 dB with a 3.5:1 ratio, and a master gain. The gain is $0.5 \cdot v^2$ for volume v. Mute wins over volume, and nothing sounds before a user gesture.

8.5 Emergency descriptor export

The seed words are the only backup, but they do not describe how a custody output is built. The wallet therefore offers **EXPORT EMERGENCY DESCRIPTORS** in settings: one plain-text JSON document (also delivered as a QR) that is enough for another taproot-capable tool to spend every staged address without this wallet.

A *staged address* is the P2TR state output of a frog. Per frog the export carries:

Listing 10: *Per-frog emergency export fields.*

```
sat, gen, dist, state, name
mint_id            "<txid>i<index>"
state_outpoint     "<txid>:<vout>"      (when known)
state_value        sats
owner_xonly        32-byte hex
internal_key       50929b74...3ac0
output_key         32-byte x-only output key of the staged address
leaves[]           { dir, script_hex, leaf_hash, control_block }
send_leaf          { script_hex, leaf_hash, control_block }
descriptor         tr(<internal>,{raw(<L>),raw(<S>),raw(<R>),raw(<X>)})#checksum
```

A Hopper on a legacy path exports the released owner; a v7-wire Hopper also uses the `FRG7CROSS` leaves. `owner_xonly` is that legacy owner. Before displaying the document the wallet rebuilds each output key and compares it with the scanned state scriptPubKey; a mismatch removes that frog from the export.

and once for the wallet:

Listing 11: *Wallet-level emergency export fields.*

```
type               "hopper-v8-emergency"
version            1
network            mainnet | testnet | signet | regtest
fingerprint        BIP-32 master fingerprint (8 hex)
owner_path         m/84'/<coin>'/0'/2/0
descriptor         tr(<internal>,{raw(<L>),raw(<S>),raw(<R>),raw(<X>)})#checksum
                   (present only when exactly one staged tree; null otherwise)
generated_at       ISO-8601 UTC
tip_height         the height the export was built at
```

Each `descriptor` line uses the BIP-380 checksum and the miniscript `raw()` fragment for that frog's four leaves. Many signers cannot parse raw leaves, so the `leaves[]` hex and control blocks are the authoritative form; the per-frog descriptor is a convenience for tools that can. The wallet-level descriptor is present only when exactly one tree is staged; with more than one it is `null`, and the per-frog descriptors are the complete set.

Rules:

- no seed words, no private keys, no xprv. The export is a *layout* backup, not a key backup: whoever holds it still needs the seeds;
- the signing key is the BIP-32 key at `owner_path`, tweaked once with `TapTweak(P)` and an empty merkle root (the BIP86 tweak); the leaf's `owner_xonly` is the resulting output key. Spending uses leaf version `0xc0` and SIGHASH_ALL (`0x01`), with the script and control block from `leaves[]`;

- a receiving tool must verify the export before trusting it: rebuild each leaf hash, each branch, the root, the output key and every control block from the export's own bytes, and compare the result with the on-chain scriptPubKey of `state_outpoint`. The wallet runs the same check before it displays the export;
- the export is public data about the wallet's holdings. It is shown behind a confirmation and is never logged, uploaded, or put in a URL;
- the export is a snapshot, and every hop, prepare or transfer changes a state outpoint. The wallet stamps `tip_height` and regenerates the export after every action; a tool that sees a state outpoint already spent must discard the entry rather than guess.

8.6 Wallet safety invariants

These are mandatory before any transaction is signed or broadcast.

1. **Complete ranges.** Every funding or state input has non-empty ordinal ranges whose lengths sum to its value. An empty ord/indexer answer is not proof that a coin has no inscription.
2. **FIFO assignment.** The wallet recomputes the sat ranges of the exact signed outputs. A transfer, prepare or hop must place the Hopper sat in output 0 at offset 0; a send must not place any known protected sat in a recipient output, change, or the fee. RBF and the approval path repeat this proof.
3. **Asset recheck.** Hop, prepare, transfer, send and mint revalidate every live input immediately before broadcast. The old exemption for send and mint is removed.
4. **State discovery.** A state script alone never proves its sat ranges. Local discovery asks ord or the indexer for complete ranges and marks the coin unspendable until they match the value and contain the frog sat.
5. **Recovery is fail-closed.** A restore paginates address histories to exhaustion, treats `more` or `null` as incomplete, reads at most a bounded checkpoint, and never synthesizes a range or a mint sat from a document.
6. **Reorg identity.** Scan gates and outcome-hash caches are keyed by $(\text{height}, \text{tip hash})$, not height alone. A same-height hash change forces a fresh pass.
7. **Vault durability.** Vault creation writes, reads back and verifies the ciphertext; a storage failure is an error. Refresh summaries carry public keys only.
8. **Service bounds.** Public reads are rate-limited, default 300 per minute per caller; diagnostic `/v1/state/root` is not a hot path; and no state lock is held across an outbound ord call.
9. **Scan freshness.** A new automatic tip gets two gate-bypassing catch-up passes (`TIP_RECHECK_MS`, about 1.2 s) so a lagging address index cannot freeze the UTXO view. A completed scan repaints the current view; if a foreground action owns the overlay, the repaint is queued and applied on release, never dropped.

8.7 The user's trust boundary

The wallet never sends the mnemonic or the sealed entropy to a server. It signs locally. It refuses a mint or a hop if the indexer's `params_hash` differs from the embedded one, if the indexer does not answer the parameter gate, or if its `network` does not match the wallet's. It refuses a non-P2TR destination for custody. It refuses to spend a coin whose ordinal ranges it cannot attribute. It never replaces a HOPPER action with RBF. These refusals are part of the wallet's safety model, not UI preferences.

The wallet is not a custodian

A wallet can be lost, replaced, or run offline. The frog remains a UTXO on Bitcoin. Recovery is the 12 words; the public ledger is a cache.

9 Security analysis

9.1 Threats and mitigations

Table 16: *Threats and protocol-level mitigations.*

Threat	Mitigation
Forge a crossing	The chosen cross leaf requires an owner signature and a canonical 55-byte body. The body binds the sat, mint txid, mint index and direction; the state output is a one-time UTXO.
Replay a witness on another frog	The body and the parent tag both name the mint; the state output commits to the owner. A copied witness fails `CHECKSIG`, the body check, or the spent-leaf check.
Replay a witness with a different direction	The direction is the last byte of the body pushed by the leaf, and the leaf hash is committed by the control block. A witness that spent the left leaf cannot authorize a straight or right body.
Counterfeit a mint	A mint requires exactly one inscription reveal in input 0, no pointer, the correct content type, the canonical v8 body digest, and a first sat that is tracked and frog-free. The v7 digest is accepted only for the grandfathered sat; v6 and every other v7 mint are rejected.
Hide a death	The fold derives the wrap from $H(T+1)$ and the direction in the body. The static band never prints a count it cannot know.
Choose the outcome	The author chooses T and the direction, not $H(T+1)$. The target is frozen at T; the wrap does not exist when the crossing is signed. A block producer can influence the candidate hash or withhold a block, but that is miner influence, not player agency.
Blame another Hopper for a block's wrap	The wrap is global and depends only on the block hash. One frog's death says nothing about another frog's sat or owner.
Kill without publishing	A transfer publishes nothing, so a frog cannot be killed without publishing a crossing. A burn is a separate, observable terminal event.
Pointer attack	HOPPER never writes a pointer tag, and a mint carrying one is rejected. The inscription stays on the anchor sat.
External-origin phishing	The mint document contains no external URL. A viewer with different recursion semantics sees the static band, not a third-party page.
Accidental burn	The wallet never uses the frog address for funding and refuses to spend a coin whose ranges it cannot attribute.
Parameter confusion	The parameter hash is committed in snapshots and checked by the wallet before mint or hop. A mismatch is a hard error. A `generation_len` that does not match its formula is rejected.
Indexer lies	The indexer is not in the art's trust path. The viewer's own ord node supplies the satpoints, bodies, raw transactions and block hashes used by the fold.
Fabricated child	A body is not believed because it names the right parent. The fold checks the child's sat and satpoint, then proves from the raw transaction that input 0 spent the canonical state leaf for the body's direction. A body pasted into a funding input, or an ordinary output-only deposit, fails the proof and renders the fold unverified.
Reorg	Timers are pure functions of height and hash. The indexer replays to the fork point; the fold simply asks again.

9.2 Properties

The following statements are informal but precise enough to test.

1. **Unforgeability.** Under the security of BIP-340 signatures on secp256k1, an adversary without the owner key cannot produce a witness that passes a cross or send leaf with a valid sighash.
2. **Direction binding.** The direction is part of the body, the body is pushed by the leaf, and the leaf hash is committed by the control block and the output key. A crossing cannot be re-labelled with another direction, and a witness cannot be replayed with one.
3. **No replay across frogs.** A state body names $(\mathrm{sat}, \mathrm{mint_txid}, \mathrm{mint_index})$, and the state output is spent once. A witness for one frog cannot satisfy another frog's state output or body check.
4. **Proof-checked public verifiability.** The fold reconstructs state from the mint document, the crossing inscriptions, the raw transactions, the tip and block hashes. A body-valid child without a valid canonical spend makes the fold unverified rather than optimistic. No secret is required and no HOPPER-specific server is authoritative.
5. **Outcome independence.** The verdict depends on $H(T+1)$, which is not known when the crossing is signed, and on the direction the owner committed. The owner can time the crossing and pick a direction; it cannot select a favorable hash.
6. **Deterministic replay.** The state machine is a pure function of blocks, seeds and parameters. Two honest indexers with the same inputs produce the same state root and event log.
7. **Fail-closed art.** If the fold cannot fetch a child, verify a satpoint or raw proof, complete pagination, or run JavaScript, the document presents the static band or the unverified band with an unknown count. It does not claim the frog is alive, and it does not claim `HOME`.

The first three properties are the custody argument. The fourth is proof-checking plus the game argument. The fifth and sixth are the infrastructure arguments. The seventh is the presentation argument.

9.3 Limitations and honest assumptions

What HOPPER does not promise

- **Liveness.** A transaction can be censored or fee-starved. The protocol decides what happens after confirmation, not when confirmation occurs.
- **Privacy.** Addresses and sat histories are public. The design is self-custodial, not anonymous. The global wrap also correlates every Hopper in a block: one block-wide coin flip, not one per frog.
- **Data availability.** The art, state bodies and raw transactions must remain retrievable from the ord node or another index. A viewer that cannot fetch them shows the fallback or unverified band.
- **Legacy reveals.** A keyless commit made before the signed commit rule remains revealable forever. No new commit may use that form.
- **Discovery.** A wallet that cannot see the chain or a trusted index may not find a frog sent to it. The frog still exists; the wallet simply has not discovered it.
- **Rule changes.** The rules are fixed for wire version `0x08`. A change requires a new wire version; it cannot be deployed by editing an indexer's configuration. The single grandfathered v7 Hopper is a custody exception, not a general compatibility mode.

- **Wrap bias.** The modulo is biased toward left by one part in 2^{32}. This is negligible for the game and is part of the consensus definition.

9.4 Why the extra transaction is unavoidable

A state leaf names the mint inscription id. The id contains the mint reveal txid. An output created by the reveal transaction cannot contain that txid, because a transaction cannot commit to its own hash. The first state output must therefore be created by a later transaction: either the first transfer or the one-time prepare. This is not an implementation artifact; it is a consequence of self-referential identity on a hash-linked ledger.

The alternative — omitting the mint id from the state leaf — would make the custody script independent of the frog's identity and would weaken replay resistance. HOPPER chooses the extra transaction and keeps the binding.

9.5 Compatibility notes

Three compatibility rules are normative and easy to misread.

1. Version 8 is the current deployment. The envelope version (`0x08`), the 55-byte body, the four-leaf tree, the proof-checked fold and the art all changed. A v8 indexer accepts exactly one v8 mint-body digest. For `GRANDFATHERED_SAT` only, the released v7 envelope, digest, `V7` body and `FRG7CROSS/0x07` state path remain valid. Every other v7 mint or crossing is rejected, and v6 bodies are rejected. The v7 card's embedded fold is immutable and predates the proof check; it is not an authoritative art oracle.
2. The indexer accepts both minimal and PUSHDATA1 spellings of short pushes in an ord envelope. The reference encoder emits minimal pushes, and byte-exact fixtures use them, but the parser is lenient in the same places ord is lenient.
3. The wrap must be computed with exactly `u32LE(H(B)_internal[0..4]) mod 3`. A wider word, a rejection scheme, or a hash of the hash changes the outcome and is not v8.

These rules do not change the game. They keep old bytes valid where they must be, and they make the decoder match the ecosystem it lives in.

10 Validation

The implementation is accepted only when the obligations in Table 17 pass. They are deliberately stated as cross-language properties, not as unit-test counts.

Table 17: *Test obligations.*

Area	Obligation
Golden vectors	JavaScript and Rust produce identical `required` across a generation boundary (5, 6, 25 for the whole hold, then 5 again), the wrap `u32LE(H)[0..4] mod 3`, the 55-byte `FRG8CROSS` body, the envelope, the parent tag, the v8 mint-body digests and the one v7 grandfather digest for pinned fixtures.
Wire bytes	The three 161-byte crossing leaves, the send leaf, the 97-byte control block for each of the four leaves, the commit leaf and the witnesses are byte-exact; parse back with the ord decoder; verify every control block against the output key.
Fold parity	The embedded fold and the indexer answer the same $(\mathrm{state}, \mathrm{dist}, \mathrm{gen})$ for the same crossing set, including a block where every frog that hopped the wrap dies and every other frog survives; a body-valid child with a missing, false or non-canonical raw proof yields *unverified*; a body pasted into a funding input or an output-only deposit cannot count; duplicate and pre-finality crossings yield *unverified*; incomplete pagination is never a lower-bound count.
Consensus	Every acceptance and rejection in Section 5, the frozen activation height and the absence of a lenient mode, the grandfathered-sat v7 path, reorgs, snapshots, the parameter hash, and a `generation_len` that does not match its formula.
Renderers	The static card with JavaScript off shows the name and the `G` badge but never a number; the proof-checked live fold shows `NAME═ N`, `RIP NAME` and `HOME` against a mocked `/r/` server; a missing or false raw proof shows `NAME ═ ---` with no count and no `HOME`; no external host in the body; the fold derives the outcome from the block hash without asking the indexer.
Emergency export	Rebuild every leaf, control block and the output key from the exported JSON alone and compare with the staged scriptPubKey; the export contains no seed, xprv or private key; a stale `tip_height` is detected.
App	Onboarding, keys, the fixed receive script, mint, prepare, the three hop choices, transfer, ordinary SEND, SEND ALL (one output, no change, fee from the swept total), history, the generation badge, display of unattributable coins, and sound mute/volume persistence; an automatic new tip owes two gate-bypassing catch-up passes, a scan repaint during a busy action is applied when the action releases the UI, and the road log distinguishes **survived** from **came home** using `immortal_height`.

10.1 Reference vectors

The following values are pinned by the reference deployment. They are regression anchors, not additional consensus constants: the consensus constants are the parameters in Section 3.

Table 18: *Reference deployment vectors.*

Item	Value
G	`974034`
Generation length L	`6480`
H_{params}	`d466f8db3efa330dc92444274675804e47ec76b62393eeae88063ef764bcf72f`
Mint body digest seed `0x9058d0fa`, sat `1000000000000`, name `Hopkinson`, `gen_hint` 1	`6ed060bc3186993e284e6655dcd082f4187ddde5b37051d8028a4e03cdea7ecf`
Mint body digest seed `0x00005eed`, sat `2099999999999`, empty name, `gen_hint` 1	`1a3ba825615d0168af07cef7fc5ccf42aed819ef6e67eb868ac099fbc10b0aa5`
Grandfathered sat	`1,965,983,893,977,665`
v7 grandfather digest seed `0x9058d0fa`, sat `1000000000000`, name `Hopkinson`, `gen_hint` 1	`18c566cc68856b41f4a71ac9bd0b39e57ed9ef25461798d8677ce490d1ee7dca`
Crossing leaf length	`161` bytes at mint index 0
Send leaf length	`34` bytes

The mint digests are for the single current `live_v8_html`. The released v7 digest is accepted only for the grandfathered sat, which is itself a regression anchor. A v6 body is not an additional accepted value; it is a different protocol.

10.2 Conformance discipline

Three habits keep the two implementations aligned.

1. Constants live in one place per language, and the parameter hash is computed from the canonical bytes rather than hard-coded.
2. Wire fixtures are generated once and checked into both test suites; no test constructs its own expected bytes from the code under test.
3. The fold and the state machine are tested against the same synthetic crossing set, so a disagreement is a protocol bug rather than a UI bug.

Test the bytes, not the prose

A specification claim that is not pinned by a byte fixture is a comment. The wire tests are the executable form of this paper.

11 Related work

HOPPER sits at the intersection of ordinal theory, inscription protocols and Taproot-based custody.

Ordinal theory and inscriptions. Ordinal theory assigns a number to every sat and defines how sats move through transactions [2, 3]. Inscriptions attach arbitrary content to a sat by committing a Taproot script containing an ord envelope and revealing it in a later transaction [3]. HOPPER adopts both ideas but uses the inscription as a state transition, not as a collectible image. The mint document is the frog's genesis; each crossing is a small binary inscription on the same sat.

Fungible token experiments. BRC-20 [11] and Runes [12] use inscriptions or OP_RETURN messages to encode balances and transfers. They rely on an indexer to agree on a ledger derived from those messages.

HOPPER has no fungible balance and no global ledger: the state is attached to one sat, and the custody rules are enforced by the script that holds it. An indexer can be wrong or unavailable without changing which frog belongs to which sat.

Taproot assets and off-chain systems. Taproot Assets [13] uses Taproot commitments to represent assets that can move on Lightning. BitVM [14] explores optimistic computation anchored in Bitcoin. HOPPER is deliberately smaller: it does not need a bridge, a channel, or a fraud proof. The game's entire outcome function is a four-byte block-hash modulo and a direction comparison, and the custody rules fit in four Taproot leaves.

Comparison. The distinguishing property is the combination of three choices: (i) identity is a sat, not a token id; (ii) every state transition is an inscription on that sat; and (iii) the spend that publishes a transition is also the spend that authorizes it, including the chosen direction. The first makes the art independent of a server, the second makes the state reconstructible from chain data, and the third makes the game self-custodial. The cost is the one-time prepare transaction and the need for ordinal-aware coin selection.

What HOPPER is not. It is not a smart-contract platform, a general-purpose asset protocol, a privacy system, or a replacement for ord. It is a game with a small, auditable consensus surface and a rendering path that degrades to an honest static card.

12 Conclusion

HOPPER makes a satoshi the protagonist of a game whose rules live on Bitcoin. A mint inscription gives the sat a name and a face. A four-leaf Taproot output gives it custody: three leaves sign a hop, one per direction, and the fourth sends it onward. A 55-byte body binds the crossing to the sat, the mint and the chosen direction, and the hash of block $T+1$ wraps to one direction for the whole block. The v8 fold proves every body-valid crossing against the raw transaction that spent the canonical leaf; when it cannot, it says *unverified* instead of guessing. The indexer and the wallet make this usable, but neither is the referee.

The design is intentionally narrow. There is no token, no bridge, no server-side game loop, and no privileged signer. The consensus surface is small enough to specify in bytes and to test across two languages: a generation sawtooth, a four-byte modulo, a direction comparison, and four fixed leaves, backed by a proof check on every counted crossing. The art is a self-contained document that can fail closed. The one unavoidable extra transaction is a consequence of self-reference on a hash-linked ledger, not an accident. The single grandfathered v7 Hopper is a custody exception carved out of already-confirmed history; it does not widen the v8 rules.

The remaining work is engineering discipline: keep the byte fixtures authoritative, keep the two implementations in lockstep, and keep the indexer's conveniences out of the trust path. If those hold, a frog is what the chain says it is. *One satoshi. One life. The chain is the referee.*

A The generation schedule

The schedule is normative and is reproduced here so the paper is self-contained. A generation runs for $L = (25 - 5) \cdot 144 + 3{,}600 = 6{,}480$ blocks.

Table 19: *Milestones inside every generation k, at offset d from $S(k) = G + (k-1)L$.*

Offset d	Required $r(S(k)+d)$	Phase
0	5	generation start
143	5	last block at the base
144	6	first growth step
2,879	24	last block below the ceiling
2,880	25	ceiling reached
6,479	25	last block of the generation
6,480 (next k)	5	reset to the base

Table 20: *The first generations of the reference deployment.*

Generation k	Start $S(k)$	End $S(k+1)-1$	Peak at $S(k)+2{,}880$
G1	974,034	980,513	976,914
G2	980,514	986,993	983,394
G3	986,994	993,473	989,874
G4	993,474	999,953	996,354
G5	999,954	1,006,433	1,002,834

A frog's generation label is fixed at mint: a frog minted in G1 is `G1` forever, even if it hops during G3. The target of a hop is $r(T)$ at the hop's confirmation height and is frozen there; a reset while the hop is in flight does not change the verdict it will be judged against.

B Deployment quick reference

Table 21: *Reference deployment values.*

Item	Value
Wire version	`0x08` (legacy `0x07` for one sat)
Activation height	`974034`
Grandfathered sat	`1,965,983,893,977,665`
Genesis height G	`974034`
Generation length L	`6480`
Generation parameters	$b = 5$, gi $= 144$, $c = 25$, $h_{\text{hold}} = 3600$
Finality κ	6 blocks
Parameter hash	`d466f8db3efa330dc92444274675804e47ec76b62393eeae88063ef764bcf72f`
State body	`FRG8CROSS`, 55 bytes (legacy `FRG7CROSS` for one sat)
Crossing leaves	161 bytes at mint index 0 (3 directions)
Send leaf	34 bytes
Control block	97 bytes
Default API bind	`127.0.0.1:9770`
Public read budget	300/min per caller (`0` disables)
Default snapshot	`hopper-indexer.snapshot.bin`
Default reorg window	288 blocks
Default snapshot interval	10 blocks

Bring-up order.

1. Start the indexer with a clean state and let it ingest from block 0. Do not set a later start height.

2. If restoring from a snapshot, verify the embedded parameter hash before serving. A mismatch is fatal, not a warning.
3. Connect bitcoind's `zmqpubhashblock` and RPC. Keep the admin token off the public network.
4. Confirm that `GET /v1/params` returns the expected `params_hash`, wire version, chain identity and network, and that `GET /v1/generation/{height}` reports the expected schedule. The wallet must refuse a missing or mismatched identity.
5. Point the wallet at the indexer or at a stock ord node. The wallet must still enforce the parameter hash and its own address rules.

Event shape. The public event envelope carries the block context and one action:

Listing 12: *Event envelope.*

```
{
  "height": 980514,
  "block_hash": "<display hex>",
  "txid": "<display hex or null>",
  "action": { "kind": "settle", "frogger_sat": 1000000000000,
              "outcome": "dead", "dir": 2, "wrap": 2 }
}
```

The four action payloads are listed in Table 7; `mint` also carries `gen`. The event log is a projection; it can be regenerated from the chain.

References

[1] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008. https://bitcoin.org/bitcoin.pdf.

[2] C. Rodarmor. *Ordinal Theory Handbook*. https://docs.ordinals.com/.

[3] The ord project. *ord: an index, block explorer and wallet for Bitcoin inscriptions*, version 0.22.2. https://github.com/ordinals/ord.

[4] P. Wuille, J. Nick, T. Ruffing. *BIP-340: Schnorr Signatures for secp256k1*. https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki.

[5] P. Wuille, J. Nick, A. Towns. *BIP-341: Taproot: SegWit version 1 spending rules*. https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki.

[6] P. Wuille, J. Nick, A. Towns. *BIP-342: Validation of Taproot Scripts*. https://github.com/bitcoin/bips/blob/master/bip-0342.mediawiki.

[7] A. Towns. *BIP-86: Key Derivation for Single Key P2TR Outputs*. https://github.com/bitcoin/bips/blob/master/bip-0086.mediawiki.

[8] M. Palatinus, P. Rusnak, A. Voisine, S. Bowe. *BIP-39: Mnemonic code for generating deterministic keys*. https://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki.

[9] P. Wuille. *BIP-32: Hierarchical Deterministic Wallets*. https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki.

[10] Blockstream. *Esplora HTTP API*. https://github.com/Blockstream/esplora/blob/master/API.md.

[11] Domo. *BRC-20 Experiment*. https://domo-2.gitbook.io/brc-20-experiment/.

[12] C. Rodarmor. *Runes*. https://docs.ordinals.com/runes.html.

[13] Lightning Labs. *Taproot Assets*. https://docs.lightning.engineering/the-lightning-network/taproot-assets.

[14] R. Linus. *BitVM: Computing on Bitcoin in Roman Style*. https://bitvm.org/bitvm.pdf.